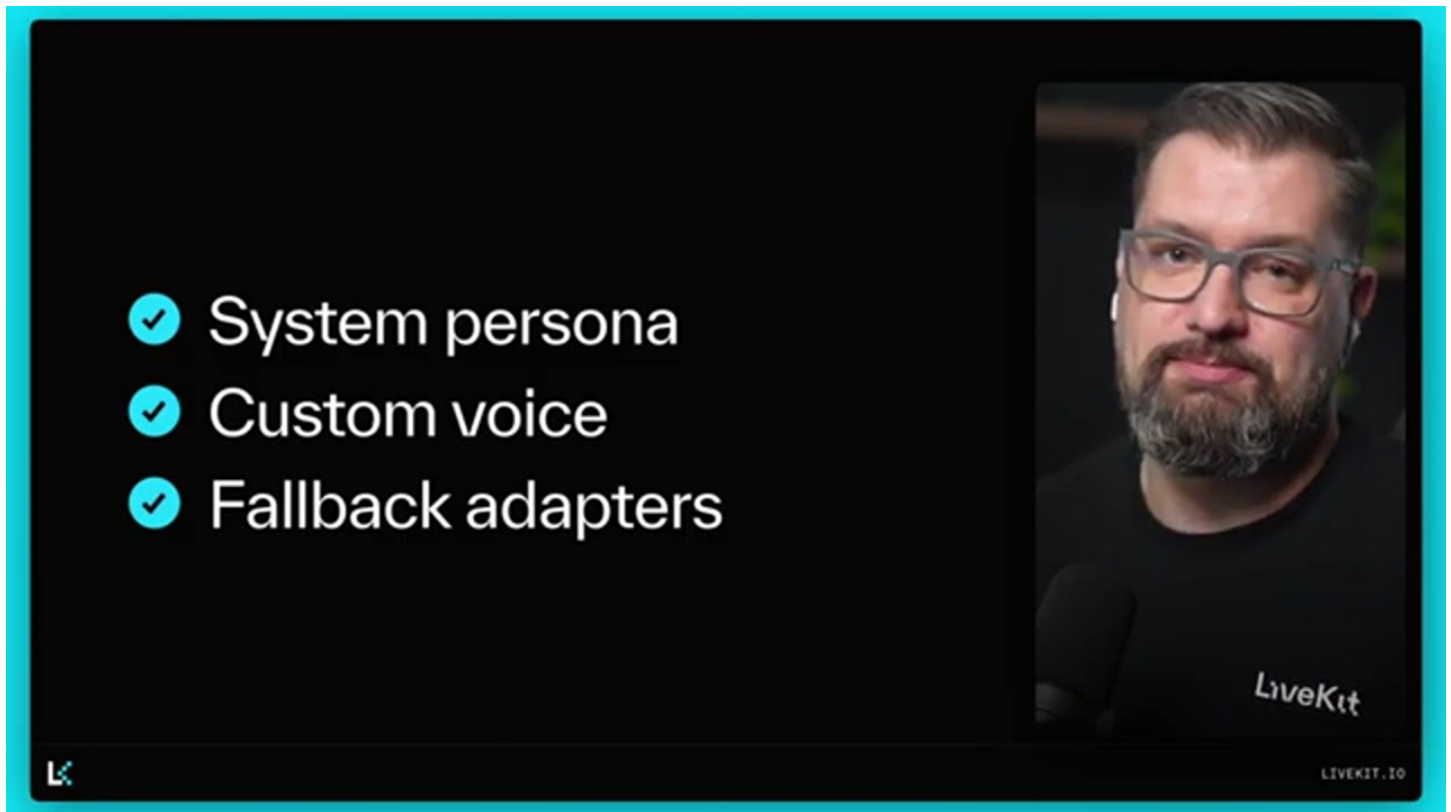


Lesson 4: Metrics, Deployment, and Latency Optimization

Section: LiveKit Voice AI Course

Summary: Coverage of Lesson 4: Metrics, Deployment, and Latency Optimization (26:11 - 44:59)

■ [26:11 - 26:31]



latency. So, your agent now sounds great and survives outages. But how do you know if it's actually performing well? This lesson covers three connected topics. Metrics collection to understand your agents behavior, deployment to LifeKit Cloud for production observability, and latency optimization techniques, including preemptive generation. So by the end you'll have

Metrics, deployment, and latency optimization

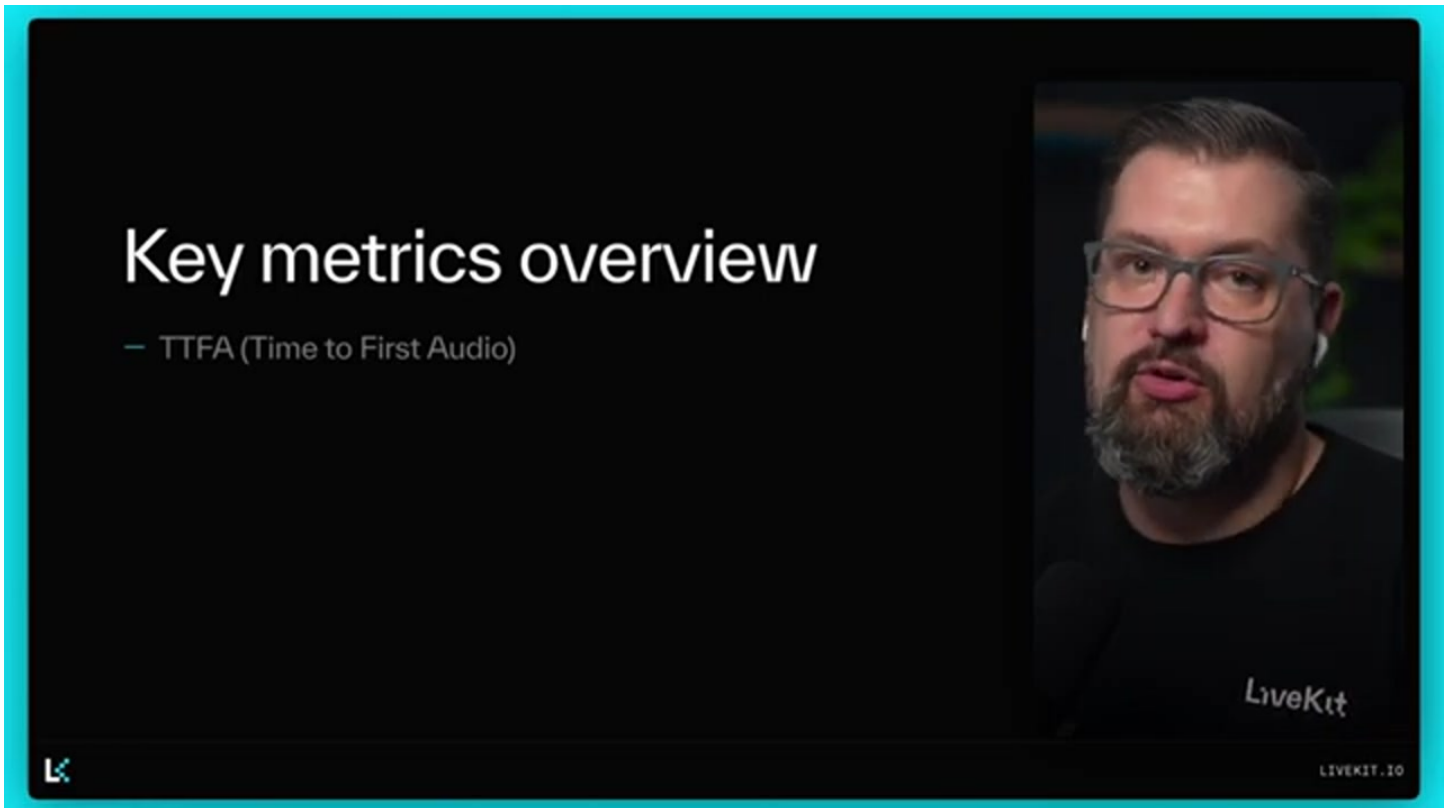


LiveKit

LIVEKIT.IO

generation. So by the end you'll have real numbers to optimize against and the tools to track them in production. Now before we jump into the code, let's talk about what you're actually measuring. So time to first audio TTFA. This is the delay from when the user stops speaking to when they hear the first word of the

■ [26:51 - 27:11]



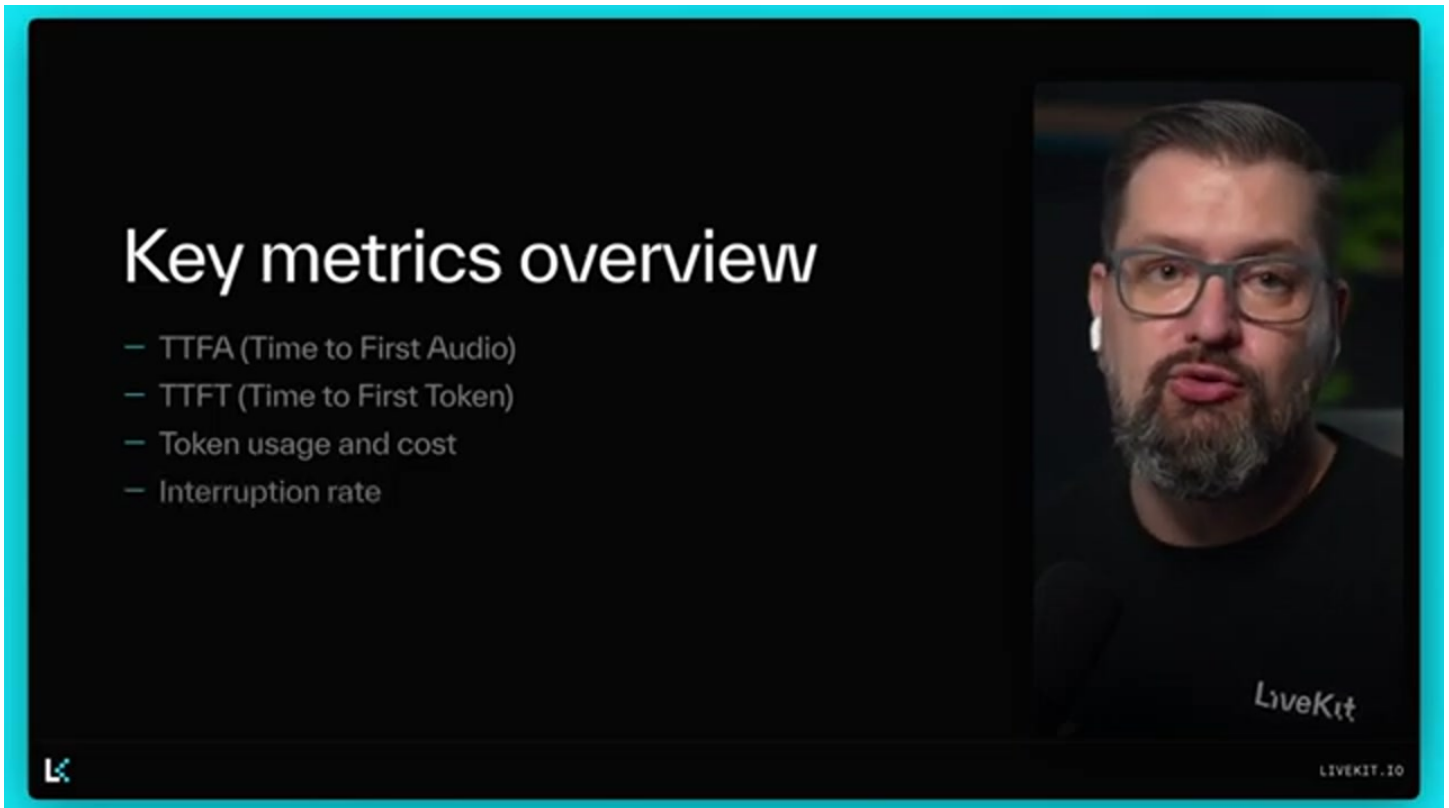
Key metrics overview

- TTFA (Time to First Audio)

LiveKit

LIVEKIT.IO

to when they hear the first word of the response. This is the number users feel most directly. And then time to first token tracks when the LLM starts generating which helps isolate whether delays are coming from transcription the model or speech synthesis. You'll also track token usage for cost estimations, interruption rate to see how often users



The video frame shows a presentation slide with a black background and white text. The title 'Key metrics overview' is at the top left. Below it is a bulleted list of four metrics. In the top right corner, there is a small video inset of a man with a beard and glasses, wearing a black shirt with the 'LiveKit' logo. The bottom left of the slide has a small 'LK' logo, and the bottom right has the text 'LIVEKIT.IO'.

Key metrics overview

- TTFA (Time to First Audio)
- TTFT (Time to First Token)
- Token usage and cost
- Interruption rate

LK LIVEKIT.IO

interruption rate to see how often users cut off the agent and fallback activations if your primary providers fail over. We're going to see these metrics show up in console logs during development and in LiveKit cloud's dashboard in production. The key takeaway here is going to be TTFA time to first audio. We want that to be less than 1 second for most responses. Let's

■ [27:31 - 27:51]

TTFA < 1000ms
for most responses



LIVEKIT.IO

than 1 second for most responses. Let's start by collecting usage metrics. The first thing we're going to do is add some imports here. So, we're going to import agent state changed event metrics collected event metrics. Uh, all that comes from LiveKit agents. And then we're just going to define our logger here. And then we'll go down to our entry point function. So just before the

■ [27:51 - 28:11]

agent.py

livekit-voice-agent > agent.py

```
16 class Assistant(Agent):
17     def __init__(self) -> None:
18
19
20
21
22
23
24 server = AgentServer()
25
26
27 @server rtc_session()
28 async def entrypoint(ctx: JobContext):
29     session = AgentSession(
30         stt=stt.FallbackAdapter(
31             [
32                 inference.STT.from_model_string("assemblyai/universal-streaming:en"),
33                 inference.STT.from_model_string("deepgram/nova-3"),
34             ]
35         ),
36         llm=llm.FallbackAdapter(
37             [
38                 inference.LLM(model="openai/gpt-4.1-mini"),
39                 inference.LLM(model="google/gemini-2.5-flash"),
40             ]
41         ),
42     )
```



entry point function. So just before the session start, we've added this code block here. So let's break this down to see what this does. The usage collector aggregates data across all conversation turns. It tracks token counts for the LLM, audio durations for ST and TTS and cost estimates when available. The metrics collected event fires after each

■ [28:11 - 28:31]

```
agent.py x
livekit-voice-agent > agent.py
28  async def entrypoint(ctx: JobContext):
52      )
53
54      usage_collector = metrics.UsageCollector()
55      last_eou_metrics: metrics.EOUMetrics | None = None
56
57      @session.on("metrics_collected")
58      def _on_metrics_collected(ev: MetricsCollectedEvent):
59          nonlocal last_eou_metrics
60          if ev.metrics.type == "eou_metrics":
61              last_eou_metrics = ev.metrics
62
63          metrics.log_metrics(ev.metrics)
64          usage_collector.collect(ev.metrics)
65
66
67      async def log_usage():
68          summary = usage_collector.get_summary()
69          logger.info("Usage summary: %s", summary)
70
71
72      ctx.add_shutdown_callback(log_usage)
```



metrics collected event fires after each component finishes processing. You're capturing uh EOU metrics specifically, which stands for end of utterance. This is the moment when the the turn detector decides that the user has finished speaking and then when the uh worker shuts down log usage prints a summary so that you can see uh per session resource

■ [28:31 - 28:51]

```
agent.py x
livekit-voice-agent > agent.py
28  async def entrypoint(ctx: JobContext):
53
54      usage_collector = metrics.UsageCollector()
55      last_eou_metrics: metrics.EOUMetrics | None = None
56
57      @session.on("metrics_collected")
58      def _on_metrics_collected(ev: MetricsCollectedEvent):
59          nonlocal last_eou_metrics
60          if ev.metrics.type == "eou_metrics":
61              last_eou_metrics = ev.metrics
62
63          metrics.log_metrics(ev.metrics)
64          usage_collector.collect(ev.metrics)
65
66
67  async def log_usage():
68      summary = usage_collector.get_summary()
69      logger.info("Usage summary: %s", summary)
70
71
72  ctx.add_shutdown_callback(log_usage)
```



that you can see uh per session resource consumption. So the agent cycles through several states during a conversation listening thinking and speaking the agent state changed event. It fires each time the agent transitions between these states. When the agent enters the speaking state, you want to know how long the human waited. So, back in our

■ [28:51 - 29:11]



long the human waited. So, back in our code, below that code block that we just added, let's add another. And then be sure to uh import time up above here. We're using that in this new code block. So, this new block here, this gives you a real number to optimize against. You can watch the console output and see

■ [29:11 - 29:31]

```
29  async def entrypoint(ctx: JobContext):
68      async def log_usage():
69          summary = usage_collector.get_summary()
70          logger.info("Usage summary: %s", summary)
71
72
73      ctx.add_shutdown_callback(log_usage)
74
75      @session.on("agent_state_changed")
76      def _on_agent_state_changed(ev: AgentStateChangedEvent):
77          if ev.new_state == "speaking":
78              if last_eou_metrics:
79                  elapsed = time.time() - last_eou_metrics.timestamp
80                  logger.info(f"Time to first audio: {elapsed:.3f}s")
81
82      await session.start(
83          agent=Assistant(),
84          room=ctx.room,
85          room_options=room_io.RoomOptions(
86              audio_input=room_io.AudioInputOptions(
87                  noise_cancellation=noise_cancellation.BVC(),
88              ),
89          ),
90      )
91
```



can watch the console output and see exactly how long users wait for each response. So, we'll see this being printed in the console after each response. Now, preemptive generation lets the LLM start forming a response before the user finishes speaking. The agent waits for a clear end of turn before actually speaking, but the thinking has already begun. We can

■ [29:31 - 29:51]

Preemptive generation



LIVEKIT.IO

thinking has already begun. We can enable this in a single line. If we go back up to our agent session, we're going to add another line here. And we're just going, it's just preemptive generation and equals true. That's it. And so this can save hundreds of milliseconds off of perceived latency, especially for longer uh user turns. The

LLM starts thinking early
→ faster responses



LIVEKIT.IO

especially for longer uh user turns. The LLM gets a head start on generating the response. But that's not to say that preemptive generation is 100% good all of the time. Preemptive generation uh trades accuracy for speed. There are a few scenarios where uh this trade-off might not work in your favor. For instance, mids sentence direction

Preemptive generation tradeoffs

- Mid-sentence direction changes



LIVEKIT.IO

instance, mid-sentence direction changes. If a user says, "Book me a flight to New York." No, actually make that Chicago. The LLM might have already committed to New York. Second, complex multi-part instructions, long requests with multiple steps can get partially answered before the full context is available. Third, high accuracy domains,

Preemptive generation tradeoffs

- Mid-sentence direction changes
- Complex multi-part instructions
- High-accuracy domains



LIVEKIT.IO

available. Third, high accuracy domains, medical, legal, or financial applications where getting it right matters more than getting it fast. But for many conversational use cases, preemptive generation works well. But if you notice that the agent is uh jumping to conclusions or misinterpreting intent, consider disabling it or tuning your turn detection thresholds. So let's

Preemptive generation tradeoffs

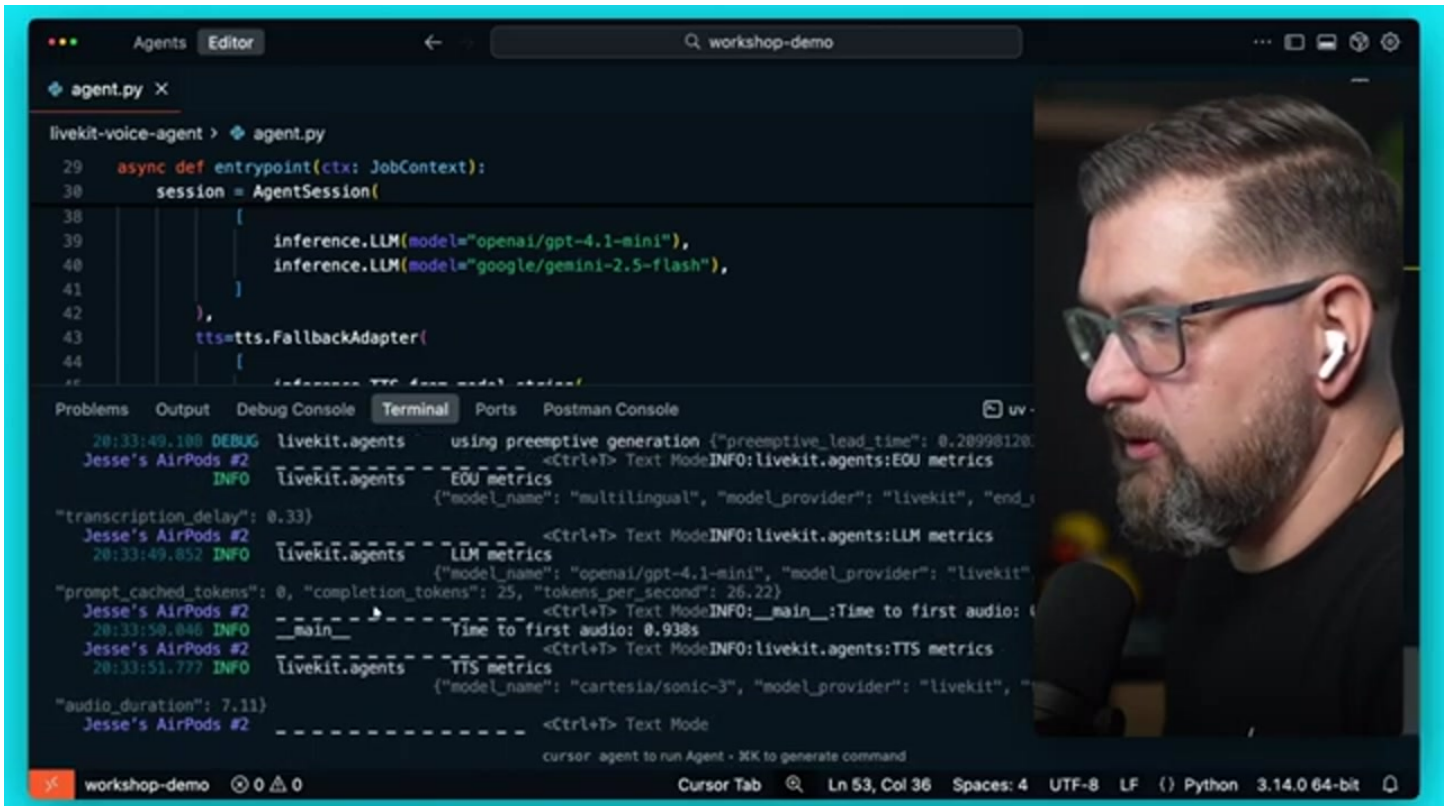
- Mid-sentence direction changes
- Complex multi-part instructions
- High-accuracy domains



LIVEKIT.IO

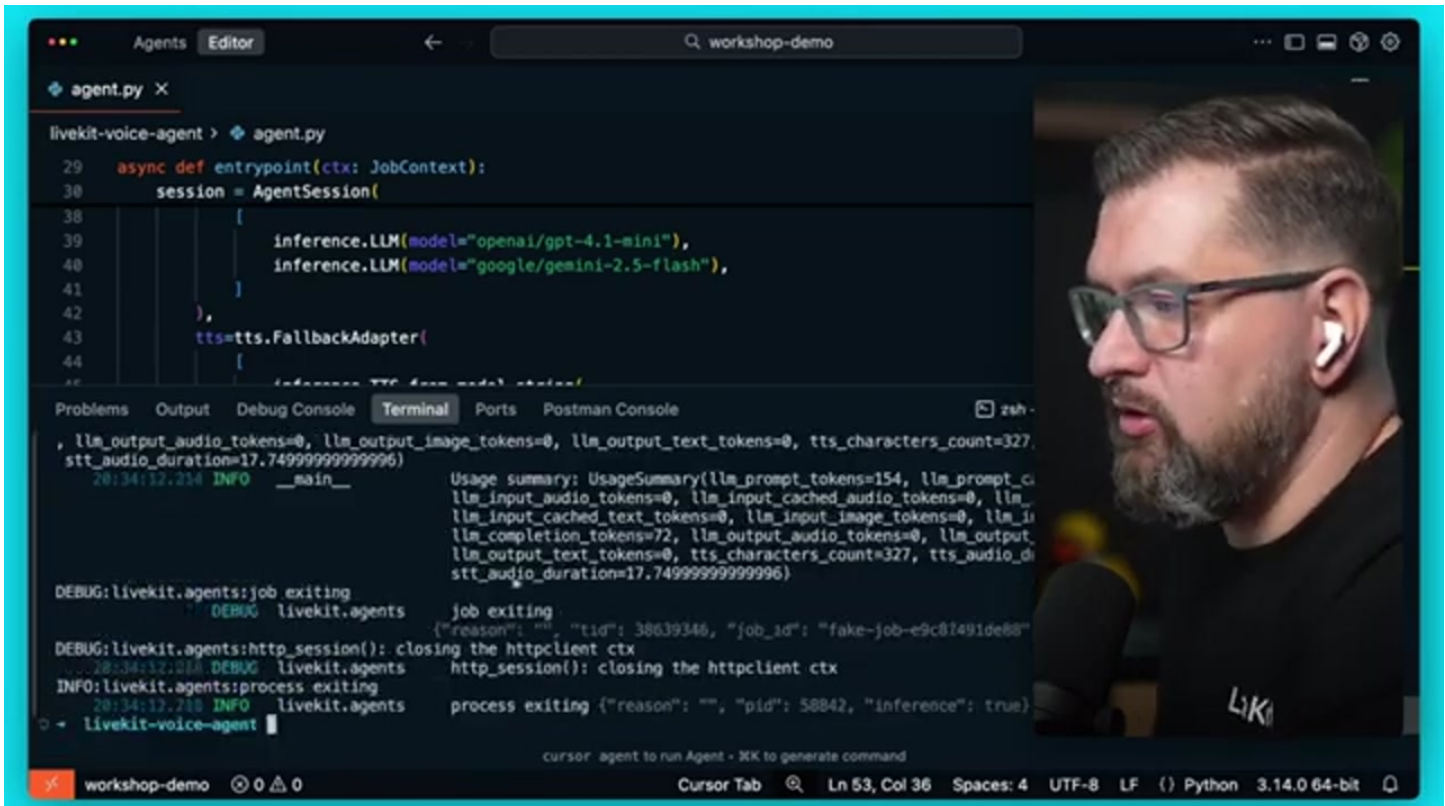
your turn detection thresholds. So let's test out these metrics and see them in action. So go ahead and open up your terminal and we'll run `uvun agent pi console`. Hello agent, how are you? Hey there. I'm great. Thanks for asking.

■ [31:11 - 31:31]



Hello agent, how are you? Hey there. I'm great. Thanks for asking. Ready to tackle your tech troubles? >> Yeah, let's see. What troubles am I having? Yeah, my my iPods have been like making crackling noises. What do you What's up with that? >> Crackling on iPods usually means dirty headphone jacks or a software glitch. >> Try clean. >> That sounds terrible. Okay, let's take a look at these metrics. So, let's go up

■ [31:31 - 31:51]



The screenshot shows a live stream interface. On the right, there is a video feed of a man with a beard and glasses, wearing a black shirt with 'LIV' on it, speaking into a microphone. On the left, there is a code editor window titled 'agent.py' and a terminal window. The code editor shows the following Python code:

```
livekit-voice-agent > agent.py
29 async def entrypoint(ctx: JobContext):
30     session = AgentSession(
31         [
32             inference.LLM(model="openai/gpt-4.1-mini"),
33             inference.LLM(model="google/gemini-2.5-flash"),
34         ],
35         tts=tts.FallbackAdapter(
36             [
37                 inference.TTS(model="openai/tts-1"),
38                 inference.TTS(model="google/text-to-speech"),
39             ]
40         )
41     )
42     # ...
43     # ...
44     # ...
```

The terminal window shows the following output:

```
Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_c
llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_
llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_i
llm_completion_tokens=72, llm_output_audio_tokens=0, llm_output
llm_output_text_tokens=0, tts_characters_count=327, tts_audio_d
stt_audio_duration=17.749999999999996)

DEBUG:livekit.agents:job exiting
DEBUG:livekit.agents:job exiting
{"reason": "", "tid": 38639346, "job_id": "fake-job-e9c87491de88"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
process exiting {"reason": "", "pid": 58842, "inference": true}
20:34:12.214 INFO __main__
20:34:12.218 INFO livekit.agents
20:34:12.218 INFO livekit.agents
```

The terminal window also shows the following output:

```
cursor: agent to run Agent - RK to generate command
workshop-demo 0 0 0
Cursor Tab Ln 53, Col 36 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

look at these metrics. So, let's go up to All right, so here we go. We've got some ST metrics here. We get the model name, the provider, and then the audio duration. Uh we've got some received user transcript here. Let's move on. ST metrics again. Received user transcript.

■ [31:51 - 32:11]

```
voice-agent > agent.py

async def entrypoint(ctx: JobContext):
    session = AgentSession(
        [
            inference.LLM(model="openai/gpt-4.1-mini"),
            inference.LLM(model="google/gemini-2.5-flash"),
        ],
        tts=tts.FallbackAdapter(
            [
                inference.TTS(model="openai/tts-1"),
                inference.TTS(model="google/text-to-speech"),
            ]
        )
    )

ms Output Debug Console Terminal Ports Postman Console zsh
se's AirPods #2 :33:56.331 DEBUG livekit.agents received user transcript
{"user_transcript": "Yeah.", "language": "en", "transcript_delay": 0.0297651290893554}
se's AirPods #2 :34:03.556 INFO livekit.agents STT metrics
{"model_name": "assemblyai/universal-streaming", "model_provider": "livekit", "audio_
se's AirPods #2 :34:03.559 DEBUG livekit.agents received user transcript
{"user_transcript": "Let's see what troubles am I having yeah my ipods have been like
what's up with that.", "language": "en", "transcript_delay": 0.45828700065612793}
se's AirPods #2 :34:03.749 DEBUG livekit.plugins.turn_detector eou prediction
{"eou_probability": 0.6739398837089539, "duration": 0.049, "input": "<|im_start|>us
u<|im_end|>\n<|im_start|>assistant\nhey there i'm great thanks for askingready to tackle your tech troubles what can i h
:|im_end|>\n<|im_start|>user\nyeah let's see what troubles am i having yeah my ipods have been like making crackling nois
```

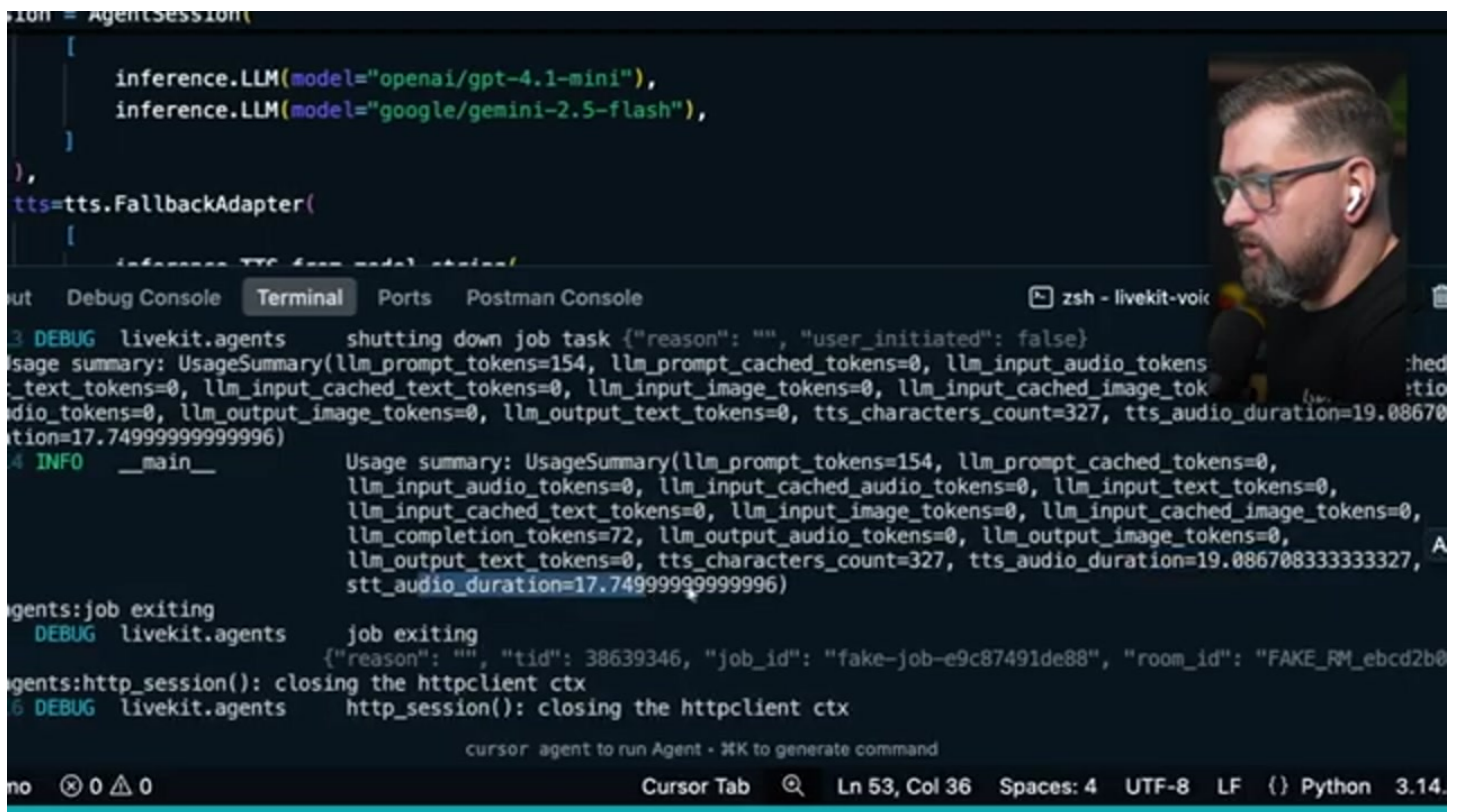
metrics again. Received user transcript. Here's some ELOU metrics here. Again, multilingual provider. Uh end of utterance delay 65. Uh time to first audio. 8889 seconds. So, a lot of great information here that we can uh use to tune our agent. Now, when we press C to

■ [32:11 - 32:31]

```
session = AgentSession(  
    inference.LLM(model="openai/gpt-4.1-mini"),  
    inference.LLM(model="google/gemini-2.5-flash"),  
),  
tts=tts.FallbackAdapter(  
    inference.TTC.from_model(stt_model,  
  
output Debug Console Terminal Ports Postman Console zsh - livekit  
.213 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}  
_Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_cached_tokens=0, llm_input_audio_token_cac  
put_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tple  
_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=19.08  
uration=17.749999999999996)  
.214 INFO _main_ Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_cached_tokens=0,  
llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_text_tokens=0,  
llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0,  
llm_completion_tokens=72, llm_output_audio_tokens=0, llm_output_image_tokens=0,  
llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=19.086708333333327  
stt_audio_duration=17.749999999999996)  
t.agents:job exiting  
DEBUG livekit.agents job exiting  
{"reason": "", "tid": 38639346, "job_id": "fake-job-e9c87491de88", "room_id": "FAKE_RM_ebcd  
t.agents:http_session(): closing the httpclient ctx  
.216 DEBUG livekit.agents http_session(): closing the httpclient ctx  
  
cursor: agent to run Agent · %K to generate command
```

tune our agent. Now, when we press C to stop our agent, uh we then get this summary here, this nice little usage summary. This shows us the total tokens used, the total audio duration processed, and estimated costs if they're available. So we can see uh completion tokens there. Uh character counts, audio duration,

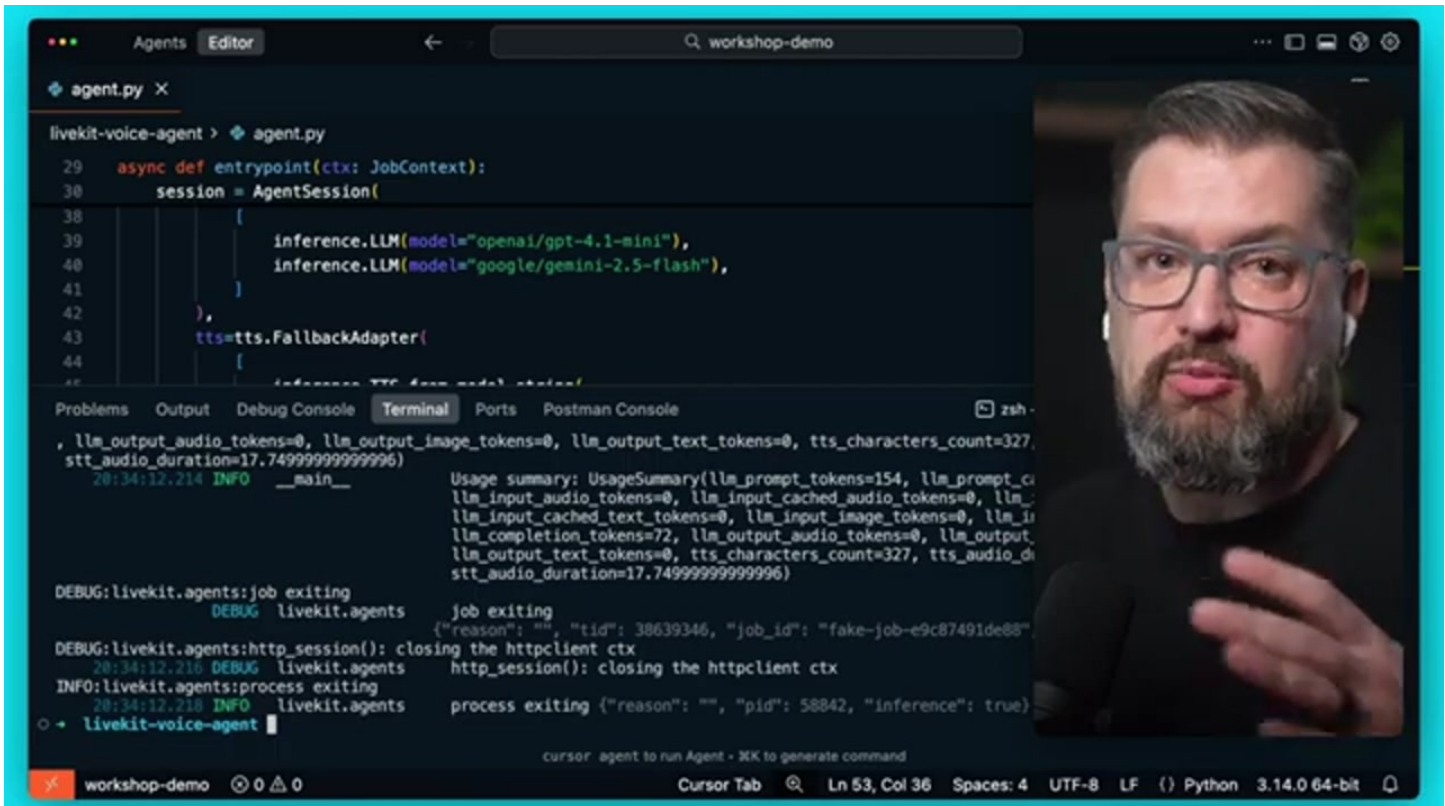
■ [32:31 - 32:51]



```
ion = AgentSession(  
    {  
        inference.LLM(model="openai/gpt-4.1-mini"),  
        inference.LLM(model="google/gemini-2.5-flash"),  
    },  
    tts=tts.FallbackAdapter(  
        {  
            inference.TTS(model="openai/tts-1"),  
            inference.TTS(model="google/text-to-speech"),  
        },  
    ),  
)  
  
3 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}  
Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_cached_tokens=0, llm_input_audio_tokens=0, llm_input_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=19.086708333333332, stt_audio_duration=17.749999999999996)  
4 INFO __main__ Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_cached_tokens=0, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_completion_tokens=72, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=19.086708333333332, stt_audio_duration=17.749999999999996)  
agents:job exiting  
DEBUG livekit.agents job exiting  
{"reason": "", "tid": 38639346, "job_id": "fake-job-e9c87491de88", "room_id": "FAKE_RM_ebcd2b0"}  
agents:http_session(): closing the httpclient ctx  
6 DEBUG livekit.agents http_session(): closing the httpclient ctx  
cursor: agent to run Agent - 3K to generate command  
no 0 0  
Cursor Tab Ln 53, Col 36 Spaces: 4 UTF-8 LF {} Python 3.14
```

counts, audio duration, ST audio duration versus TTS audio duration, LLM prompt tokens 154. So you now have visibility into your agents performance. The metrics show where time is spent, how many tokens are consumed, and how much audio is processed. So use this data to identify bottlenecks and optimize your pipeline. Remember this

■ [32:51 - 33:11]



The screenshot shows a video call interface. On the right, a man with a beard and glasses is speaking. On the left, a terminal window titled 'workshop-demo' displays the following content:

```
agent.py X
livekit-voice-agent > agent.py
29  async def entrypoint(ctx: JobContext):
30      session = AgentSession(
38          [
39              inference.LLM(model="openai/gpt-4.1-mini"),
40              inference.LLM(model="google/gemini-2.5-flash"),
41          ],
42      ),
43      tts=tts.FallbackAdapter(
44          [
45              inference.TTS(model="openai/tts-1"),
46          ],
47      ),
48  )
49  return session.run()
50
51 , llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327,
52 stt_audio_duration=17.749999999999996)
53 20:34:12.214 INFO __main__
54
55 Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_completion_tokens=72, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_text_tokens=0, llm_completion_tokens=72, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=17.749999999999996)
56
57 DEBUG:livekit.agents:job exiting
58     DEBUG livekit.agents
59
60 DEBUG:livekit.agents:http_session(): closing the httpclient ctx
61     DEBUG livekit.agents
62
63 INFO:livekit.agents:process exiting
64     INFO livekit.agents
65
66 20:34:12.218 INFO livekit.agents
67     process exiting {"reason": "", "pid": 58842, "inference": true}
68
69 ○ + livekit-voice-agent
```

At the bottom of the terminal, there is a status bar showing 'workshop-demo', 'Cursor Tab', 'Ln 53, Col 36', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.

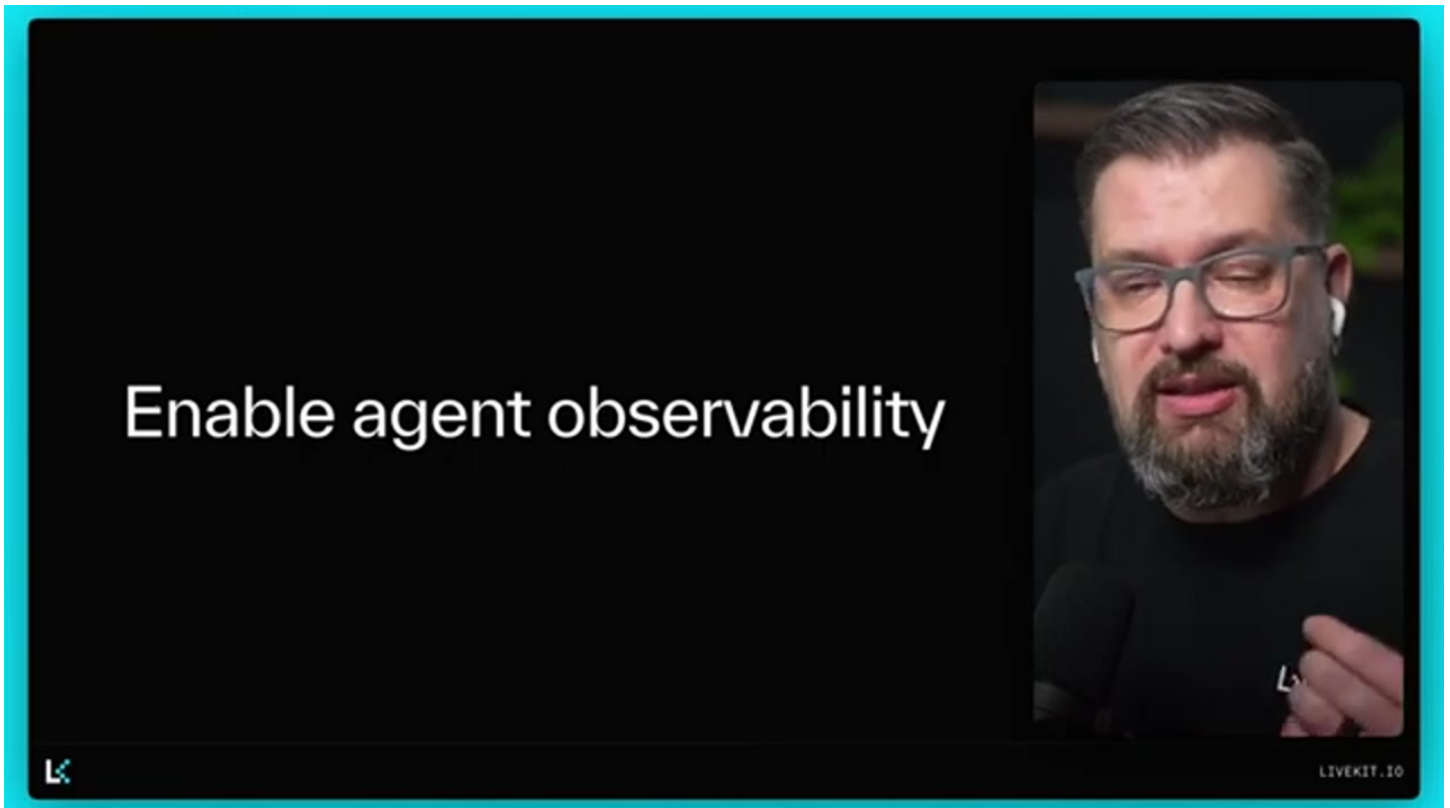
optimize your pipeline. Remember this table from lesson one. With a best case total around 400 milliseconds, but a typical latency around 1 or two seconds, any TTFA under 1 second is solid. So, you're seeing about 800 milliseconds here, which puts this response in a good territory. Now, local testing and

Enable agent observability



LIVEKIT.IO

territory. Now, local testing and metrics is fine at first. Uh, but you'll want something much more robust in production. And so, you can use external tracing tools like Langfuse if you want. Uh, but LifeKit Cloud includes built-in observability for agent sessions. So, you can view transcripts, traces, logs, and even audio recordings directly in the LifeKit cloud dashboard without any



the LifeKit cloud dashboard without any uh additional steps. LiveKit's observability is also open telemetry compatible. So, you can also export traces to any open telemetry compatible backend if you prefer. Our dashboard here shows three synchronized views. The the transcripts with audio playback let

■ [33:51 - 34:11]

The screenshot displays the LiveKit web interface. On the left is a dark sidebar with navigation links: Overview, Sessions (highlighted), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. Below these are Search and Support buttons. The main content area has a header with 'codestackr / Sessions / RM_Y7xvbEjr7okD' and a 'Download data' button. Below the header are tabs for 'Session analytics' and 'Agent insights' (which is active). A timeline at the top shows a play button, a progress bar from 00:00:00 to 00:00:15.72, and buttons for Transcript, Trace, and Logs. The transcript area shows a list of messages: a 'Session options' message, an 'Agent handoff to: assistant' message, a 'User' message at 00:03 saying 'Hello' with 99.9% transcript confidence, and an 'Agent' message at 00:05 saying 'Hey there! What tech trouble are we tackling today?' with 100% latency. On the right side of the interface is a video feed of a man with a beard and glasses, wearing a black shirt, speaking into a microphone.

the transcripts with audio playback let you scrub through the conversation. You can hear exactly what happened. The traces tab breaks down each turn into spans for uh ST, LLM, TTS, and tool calls with detailed timestamps and durations. And then logs shows real-time messages from your agent server in chronological order. All of this data is

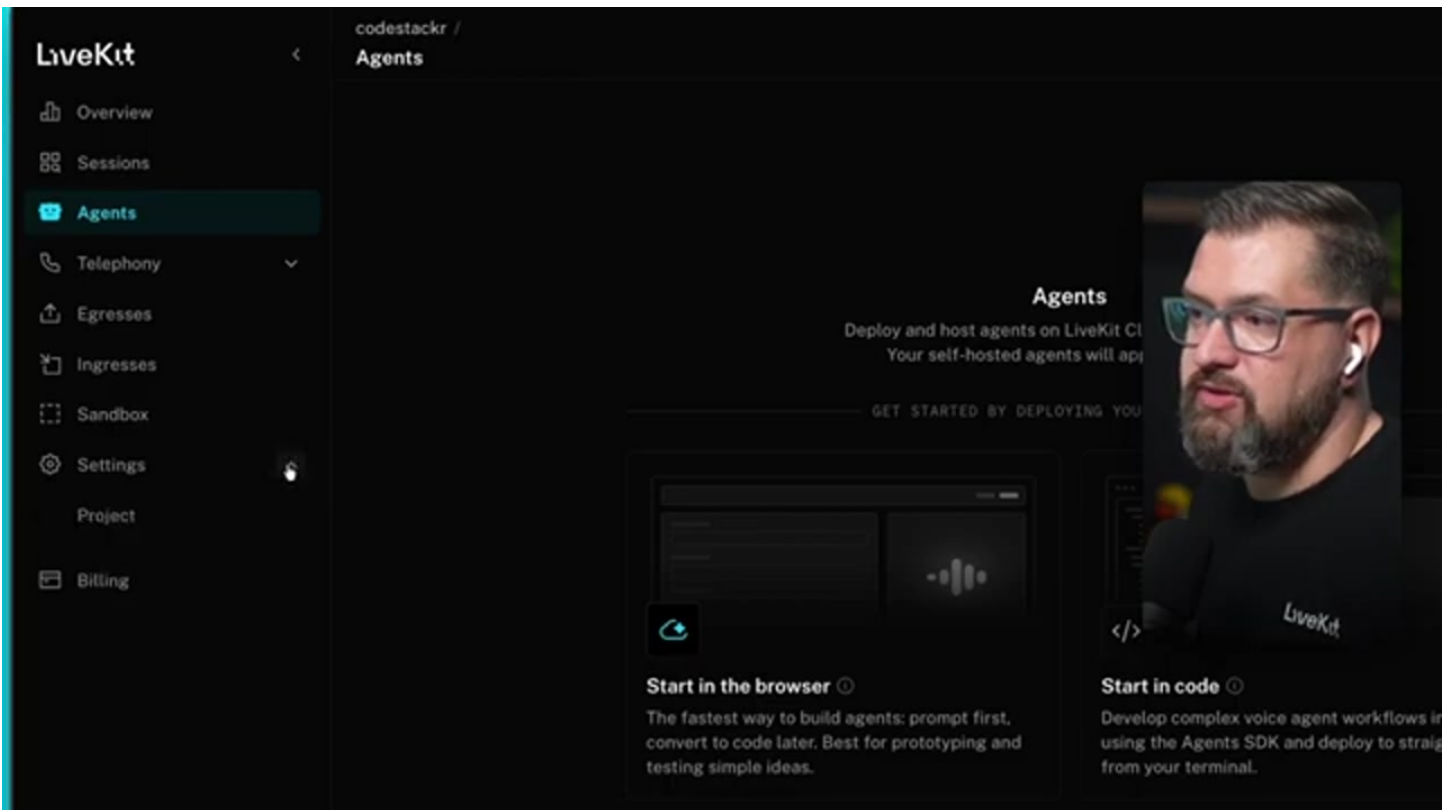
■ [34:11 - 34:31]

The screenshot displays the LiveKit dashboard interface. On the left is a sidebar with navigation links: Overview, Sessions (selected), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main content area is titled 'codestackr / Sessions / RM_Y7xvEjr7okD'. It features a 'Session analytics' tab and an 'Agent insights' tab (highlighted in blue). Below these tabs is a timeline visualization with a play button and a time range of 00:00:00:00:15.72. A table of logs is visible, with columns for Timestamp, Level, Logger, and Message. The logs show various metrics and events from Dec 26, 00:52:52.837 PM to Dec 26, 00:53:04.151 PM. On the right side of the dashboard, there is a video feed of a man with a beard and glasses, wearing a LiveKit-branded shirt, speaking into a microphone.

Timestamp	Level	Logger	Message
Dec 26, 00:52:52.837 PM	INFO	livekit.agents	STT metrics
Dec 26, 00:52:53.369 PM	INFO	livekit.agents	EOU metrics
Dec 26, 00:52:53.632 PM	INFO	livekit.agents	LLM metrics
Dec 26, 00:52:53.971 PM	INFO	...mp_main...	Time to first audio: 0.602s
Dec 26, 00:52:54.809 PM	INFO	livekit.agents	TTS metrics
Dec 26, 00:53:00.312 PM	INFO	livekit.agents	closing agent session due to participant disconnect id
Dec 26, 00:53:04.151 PM	INFO	...mp_main...	Usage summary: UsageSummary{lm_prompt_tokens:

chronological order. All of this data is synchronized to a single timeline which makes it so easy and so simple to see exactly what is happening under the hood. Now observability needs to be enabled in order to use it and we can do this at the project level. So go to your LiveKit dashboard and then under settings we'll go to

■ [34:31 - 34:51]



and then under settings we'll go to project and then we can scroll down to data and privacy and then agent observability and just make sure that that is enabled and then go ahead and save your changes. Now once it's enabled all agent sessions will automatically record data. The agents SDK version uh for Python needs to be at least 1.3.0 or

■ [34:51 - 35:11]

codestackr / Settings / Project

Save changes

LiveKit

- Overview
- Sessions
- Agents
- Telephony
- Egresses
- Ingresses
- Sandbox
- Settings
- Project**
- Team members
- API keys
- Webhooks
- Billing
- Search
- Support

Data and privacy
Configure session recording and telemetry for debugging and monitoring

Enabled codecs
Select which media codecs your project should allow.

- ☒ Opus
- ☒ Audio RED (REDundant encoding)
- ☒ H264
- ☒ VP8
- ☒ VP9
- ☒ AV1

Connection limits
To ensure the stability of our network and to prevent abuse, LiveKit Cloud projects have limitations on the number of concurrent connections. Service may be disrupted when these limits are exceeded. If you anticipate needing more concurrent connections and/or inference credits for your project, [upgrade to the Scale plan](#).

Type	Limit	Peak usage (past 7 da
Concurrent participants	100	0
Concurrent Egress requests	2	0

Agent observability
Capture traces, transcripts, and audio for hosted and self-hosted agents. [See...](#)

Enabled

for Python needs to be at least 1.3.0 or higher or 1.0.18. 0.18 or higher for NodeJS and recording happens automatically. You don't need to change your agent code at all. Audio is collected locally and then uploaded after the session ends along with transcripts and traces. Now, to view observability data in the dashboard, you

codestackr / Settings / Project

LiveKit

Overview
Sessions
Agents
Telephony
Egresses
Ingresses
Sandbox
Settings
Project
Team members
API keys
Webhooks
Billing
Search
Support

Data and privacy
Configure session recording and telemetry for debugging and monitoring

Enabled codecs
Select which media codecs your project should allow.

Connection limits
To ensure the stability of our network and to prevent abuse, LiveKit Cloud projects have limitations on the number of concurrent connections. Service may be disrupted when these limits are exceeded. If you anticipate needing more concurrent connections and/or inference credits for your project, [upgrade to the Scale plan](#).

Type	Limit	Peak usage (past 7 days)
Concurrent participants	100	0
Concurrent Excess requests	2	0

Save changes

Agent observability
Capture traces, transcripts, and audio for hosted and self-hosted agents. [See...](#)
Enabled

Opus
Audio RED (REDundant encoding)
H264
VP8
VP9
AV1

LiveKit

observability data in the dashboard, you will need to deploy your agent to LifeKit Cloud. Running locally is again fine for development, but we're at a place now where we want to start observing the performance of our agent. The decisions that we make from now on are going to have impacts on our agent and we want to know if a tool that we're adding or some feature that we're adding is introducing some poor performance.

■ [35:31 - 35:51]

LiveKit

codestackr / Settings / Project

Save changes

Allow pausing videos when subscribers are muted

Dashboard

- ✓ Show onboarding instructions on first session
- ✓ Suggest new users to directly join sessions

Agent observability

Capture traces, transcripts, and audio recordings for hosted and self-hosted agents. [See more](#)

Enabled

Data and privacy

Configure session recording and telemetry for debugging and monitoring

Enabled codecs

Select which media codecs your project should allow.

- ✓ Opus
- ✓ Audio RED (REDundant encoding)
- ✓ H264
- ✓ VP8
- ✓ VP9
- ✓ AV1

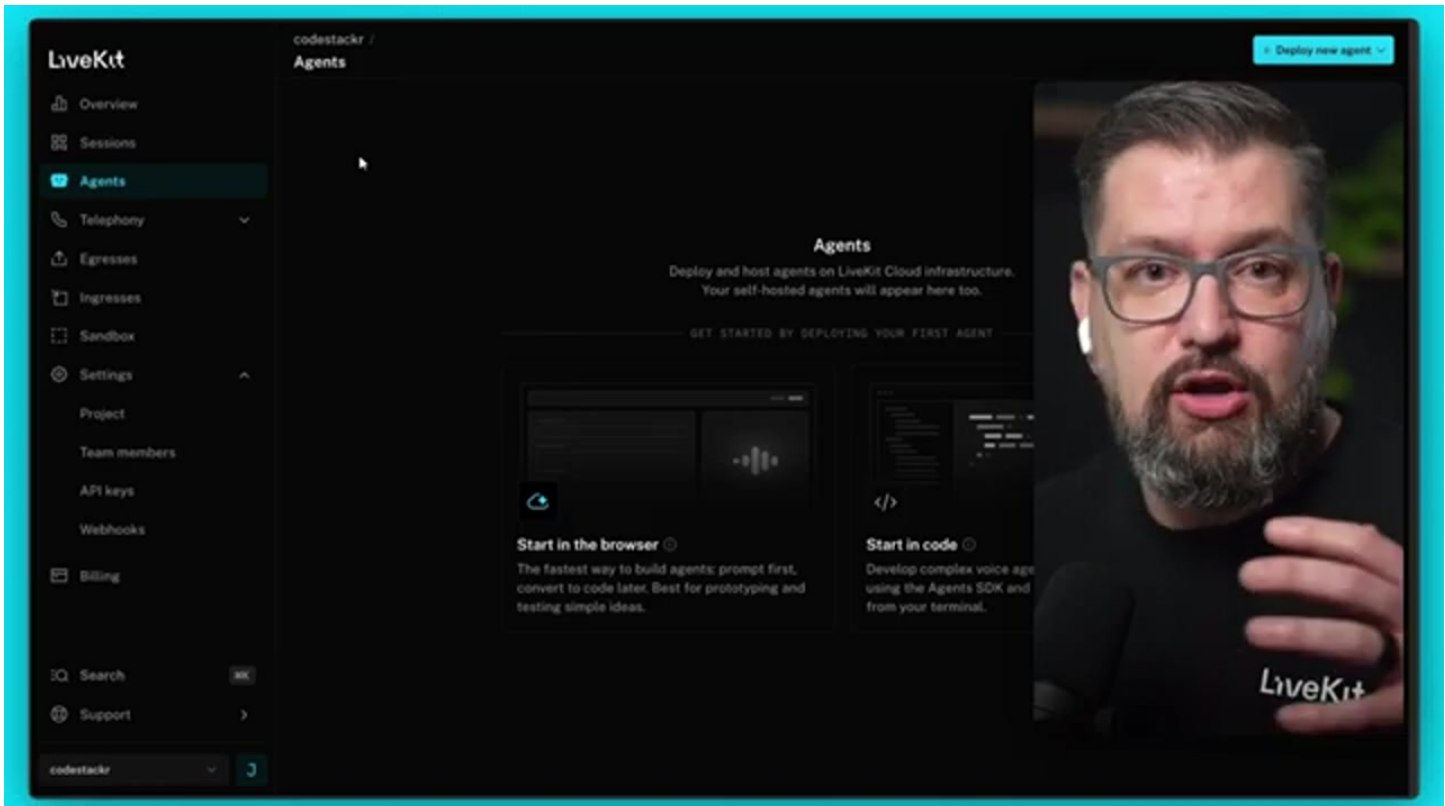
Connection limits

To ensure the stability of our network and to prevent abuse, LiveKit Cloud projects have limitations on the number of concurrent connections. Service may be disrupted when these limits are exceeded. If you anticipate needing more concurrent connections and/or inference credits for your project, [upgrade to the Scale plan](#).

Type	Limit	Peak usage (past 7 da)
Concurrent participants	100	0
Concurrent Egress requests	2	0

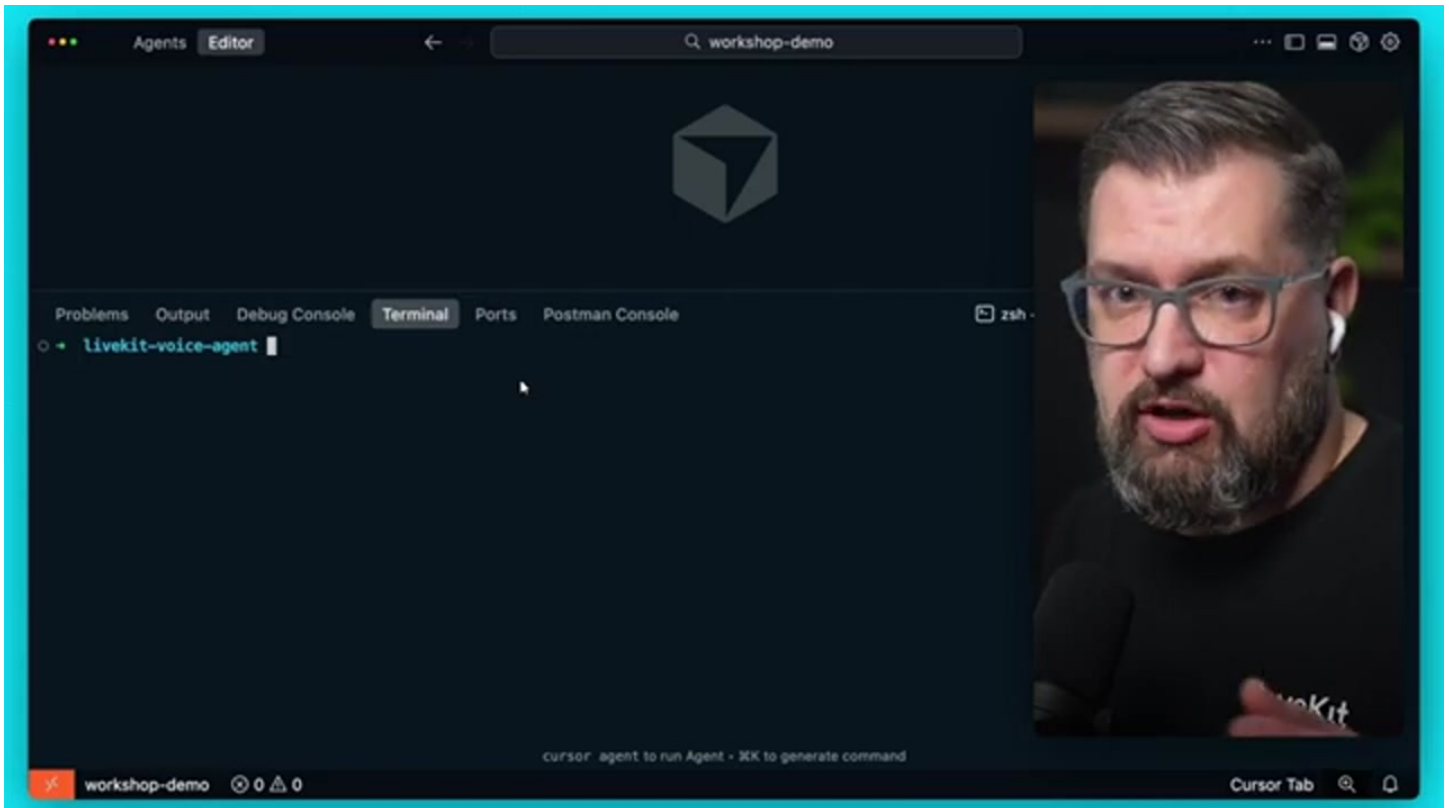
is introducing some poor performance. Now deploying to LifeKit Cloud gives you automatic scaling and load balancing. So your agent can handle multiple sessions without manual configuration. And the built-in observability integration means that transcripts, traces, and audio recordings upload automatically. Your agent runs on LifeKit's global network

■ [35:51 - 36:11]



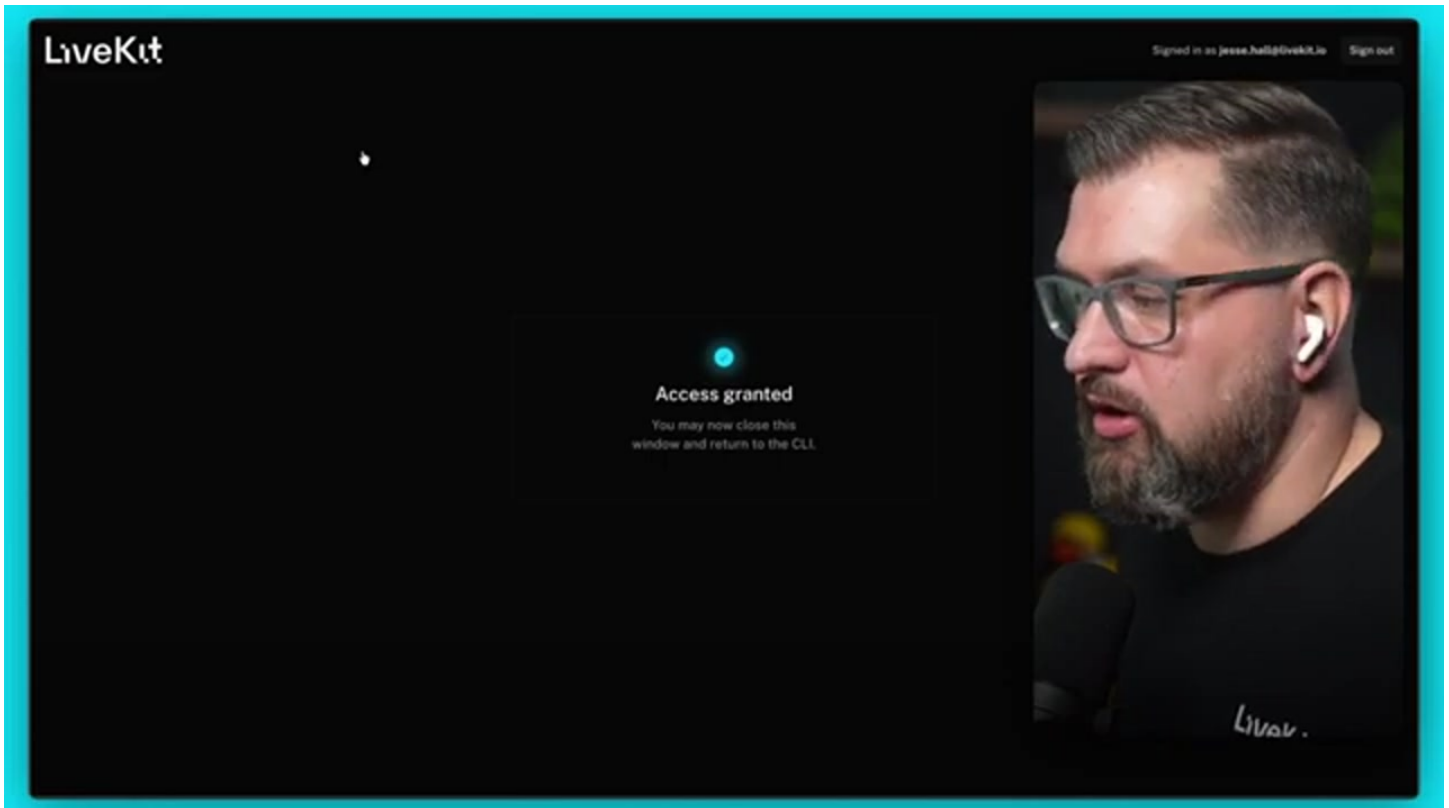
agent runs on LifeKit's global network infrastructure, reducing latency for users worldwide, and you don't need to manage servers or containers yourself. So, it's a win-win. First, you'll need to make sure that you have the LifeKit CLI installed. If you don't have it uh installed yet, you can install it with Homebrew on Mac OS, or you can download the binary from the LifeKit documentation. There's a link in the

■ [36:11 - 36:31]



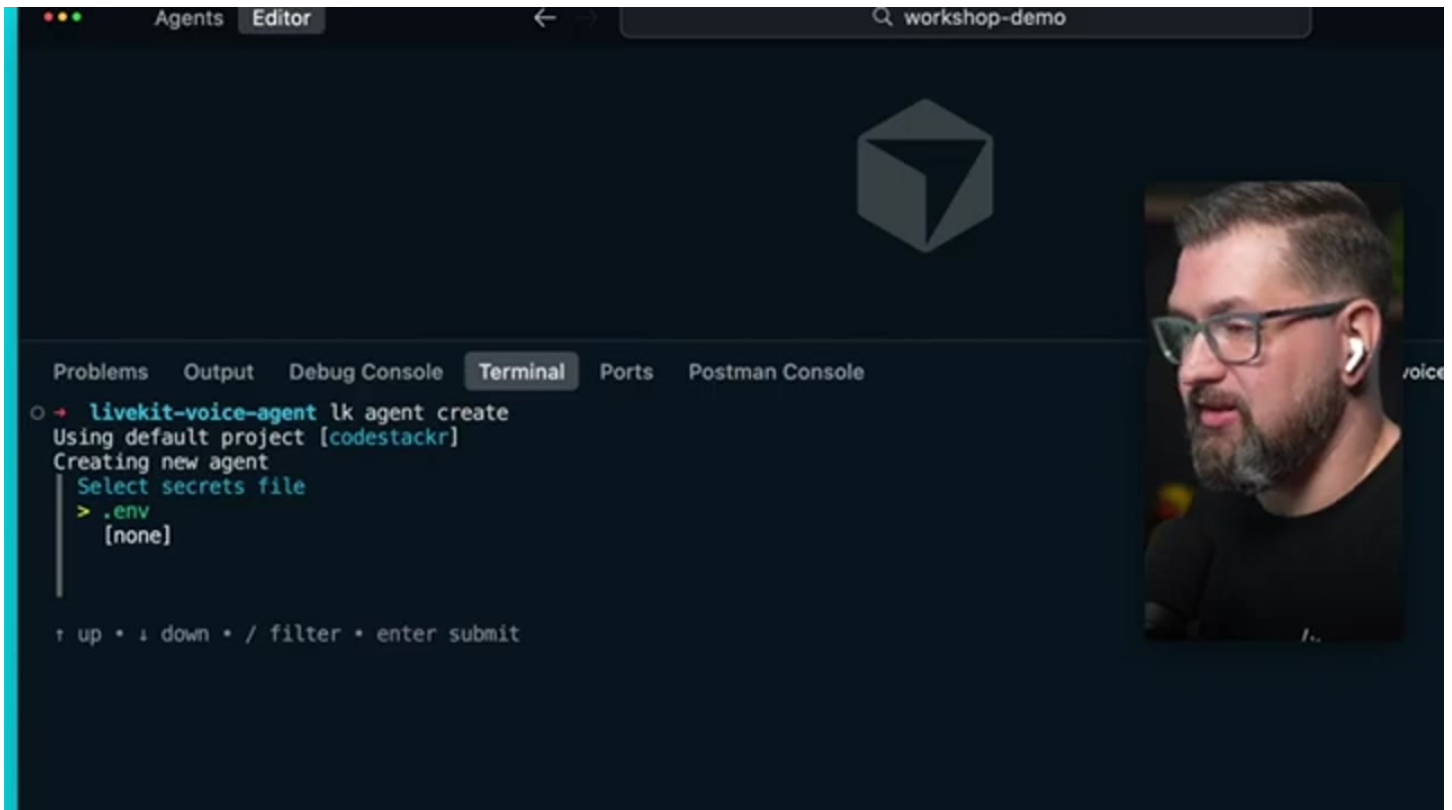
documentation. There's a link in the video description. So once you have that installed, uh we'll do LifeKit Cloud O. So that's going to pop us over to LifeKit Cloud and ask us to confirm and we'll go ahead and select a project that we want to give it access to and we're going to hit allow access. Cool. Now we can go back to our uh code editor. So

■ [36:31 - 36:51]



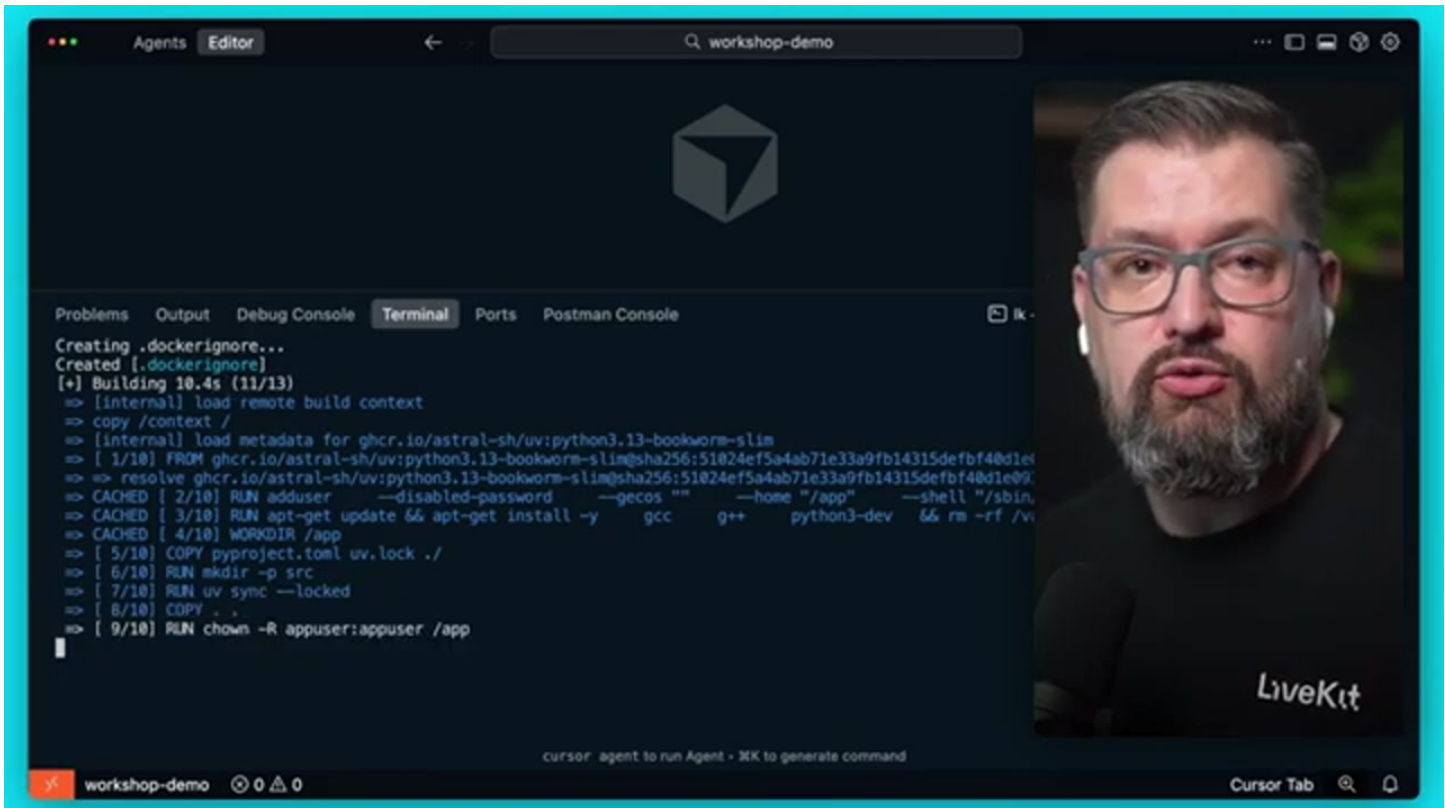
can go back to our uh code editor. So now deploying our agent is as simple as a single command. Okay. Agent create. You can specify which project if you have multiple, but I'll just go ahead and select this project. Yes, select the secrets. I already have my environment variables set up. Uh if you didn't already have them, then uh it would create them for you. So, we'll go ahead and select the existing environment

■ [36:51 - 37:11]



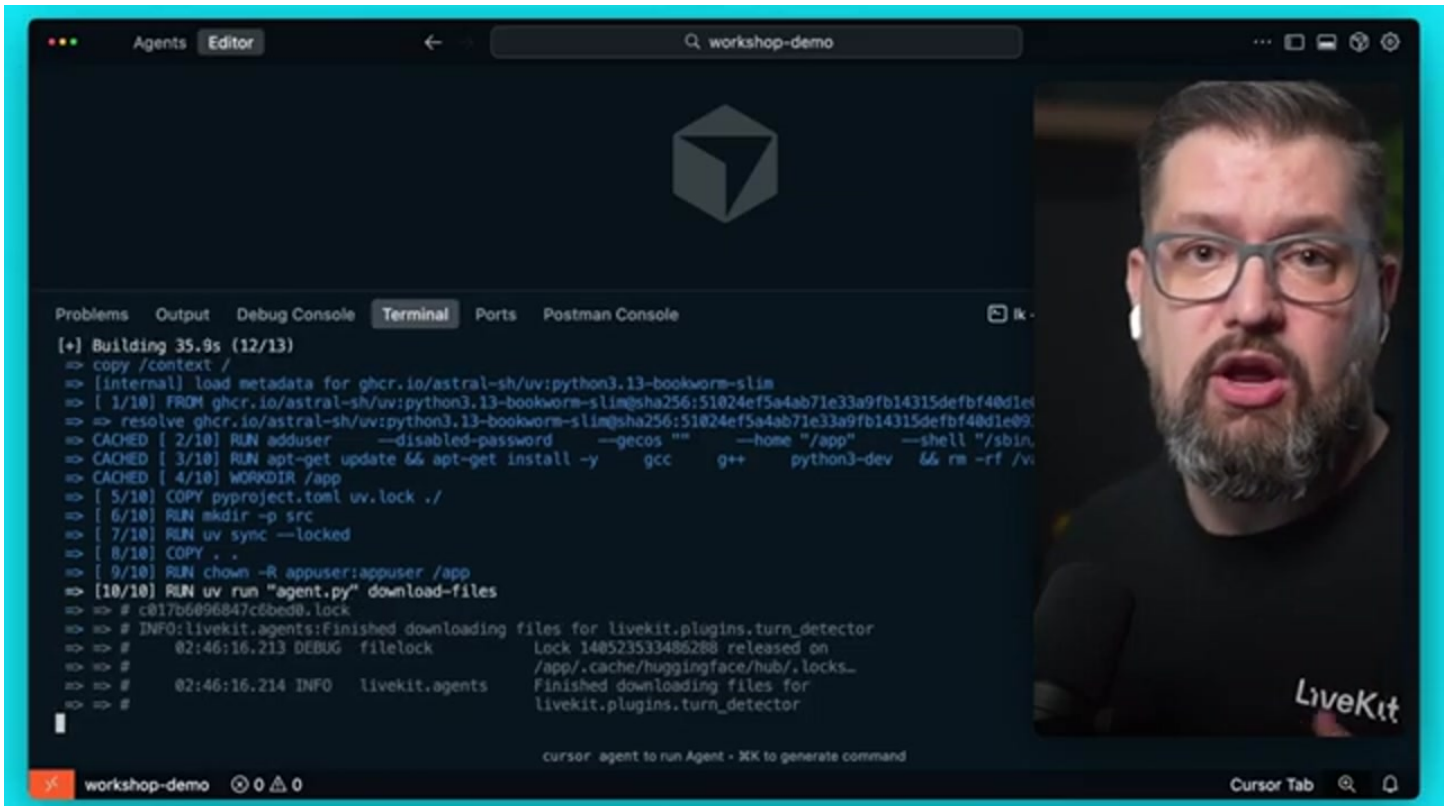
and select the existing environment variables. Uh we can select a region, US East or EU Central. So, I'll do US East. So, this command it it does several things. It registers your agent with LifeKit cloud and assigns a unique ID. The ID and the project configuration are written to a new LifeKit toml file. And if you don't already have a Docker file,

■ [37:11 - 37:31]



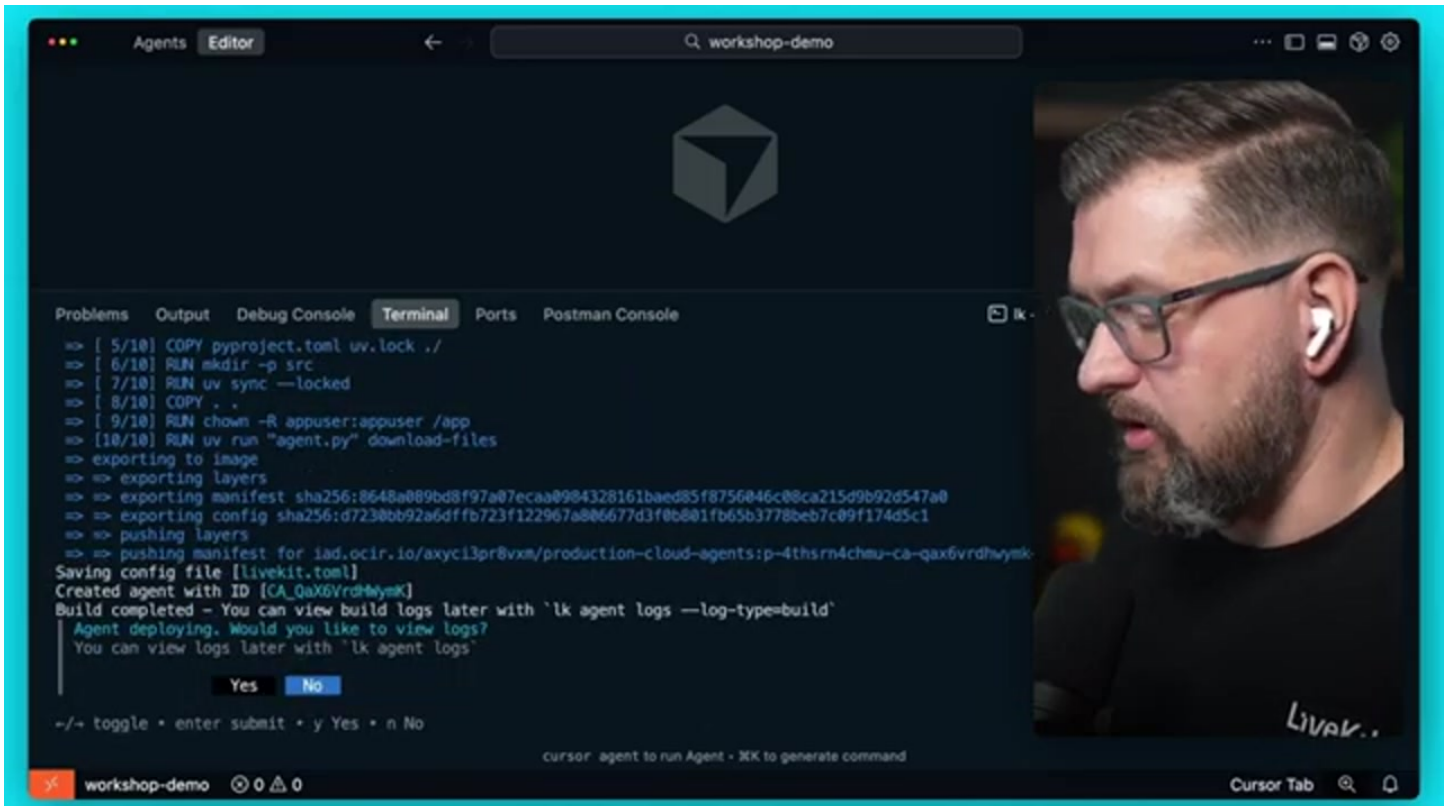
if you don't already have a Docker file, the CLI is going to create that for you as well. And then it uploads your code to LifeKit Cloud's build service, builds a container image, and then deploys it to your project. So you can see the build logs here streaming to our terminal uh as the the CA container image is creating. And this usually just takes a minute or two depending on your

■ [37:31 - 37:51]



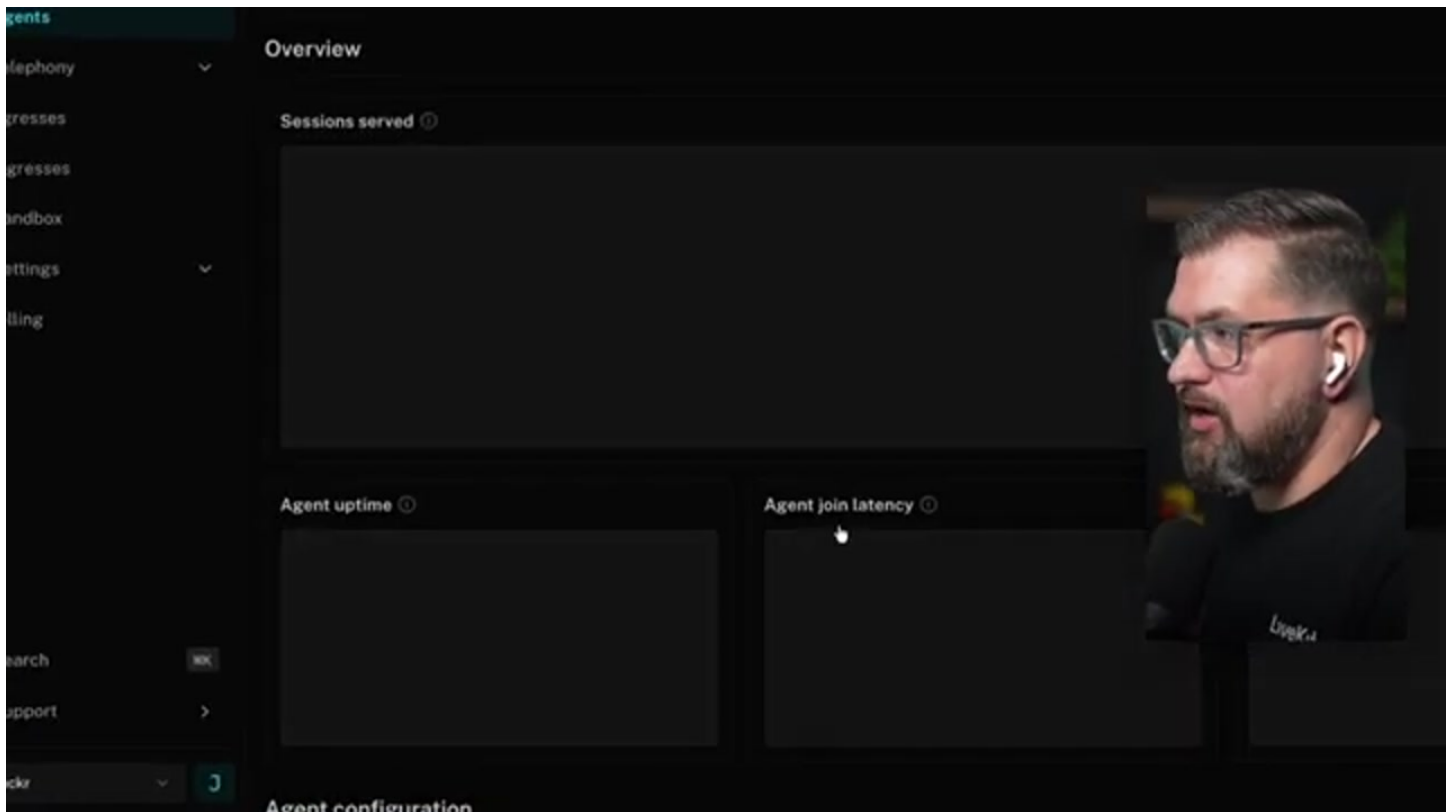
takes a minute or two depending on your dependencies. Now, once the deployment is complete, your agent is going to be live on LifeKit Cloud and ready to handle sessions. The agent automatically scales up and down based on demand. And all of the observability data will upload to the dashboard after each session. All right. So now it's asking me if I want to view logs. No, I don't

■ [37:51 - 38:11]



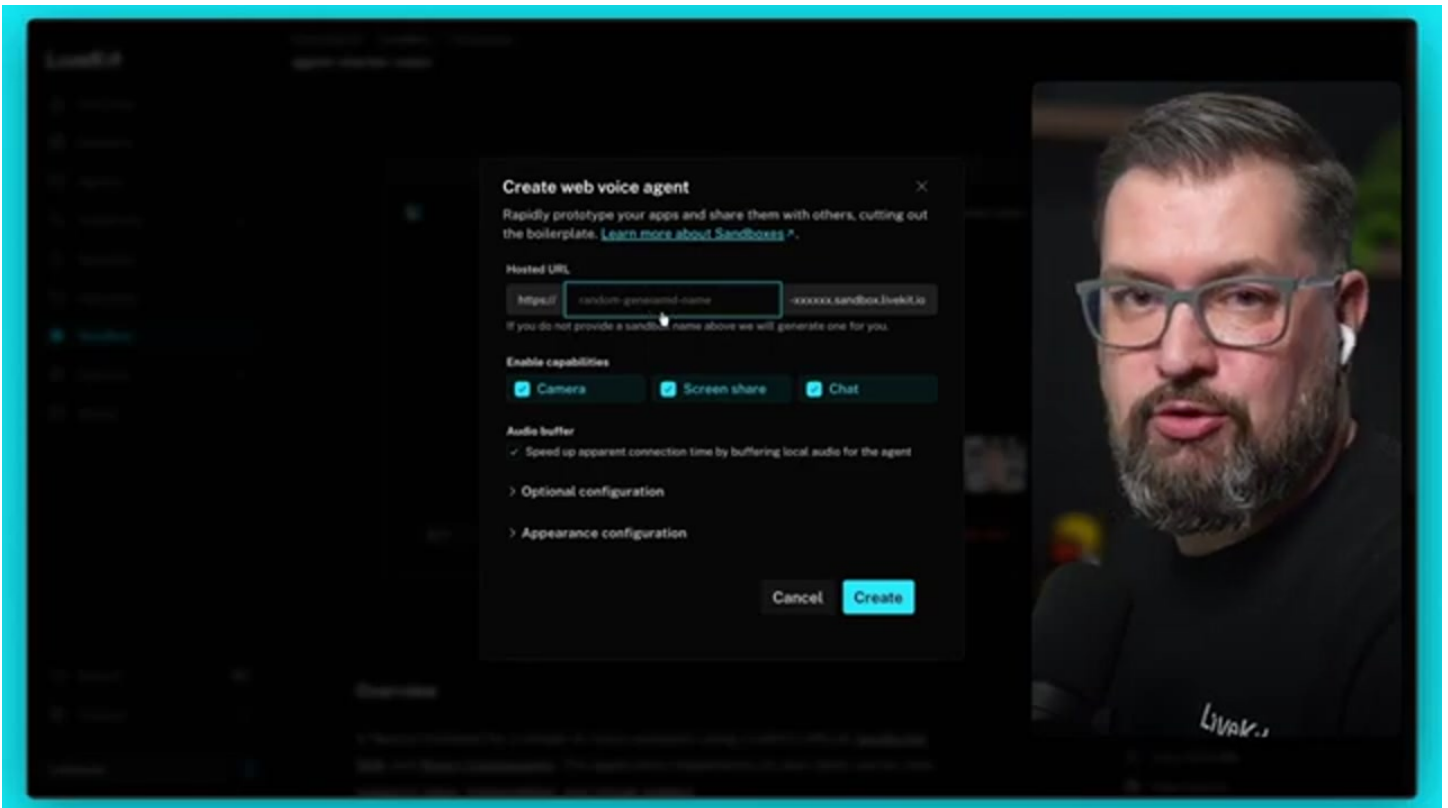
me if I want to view logs. No, I don't care about logs. And we're done. So now let's test the deployed agent to generate some session data. So notice here we didn't have anything under agents. Let me refresh the page. And now we can see our new agent. It's here. So we can get an overview of our agents here. If we click on our agent,

■ [38:11 - 38:31]



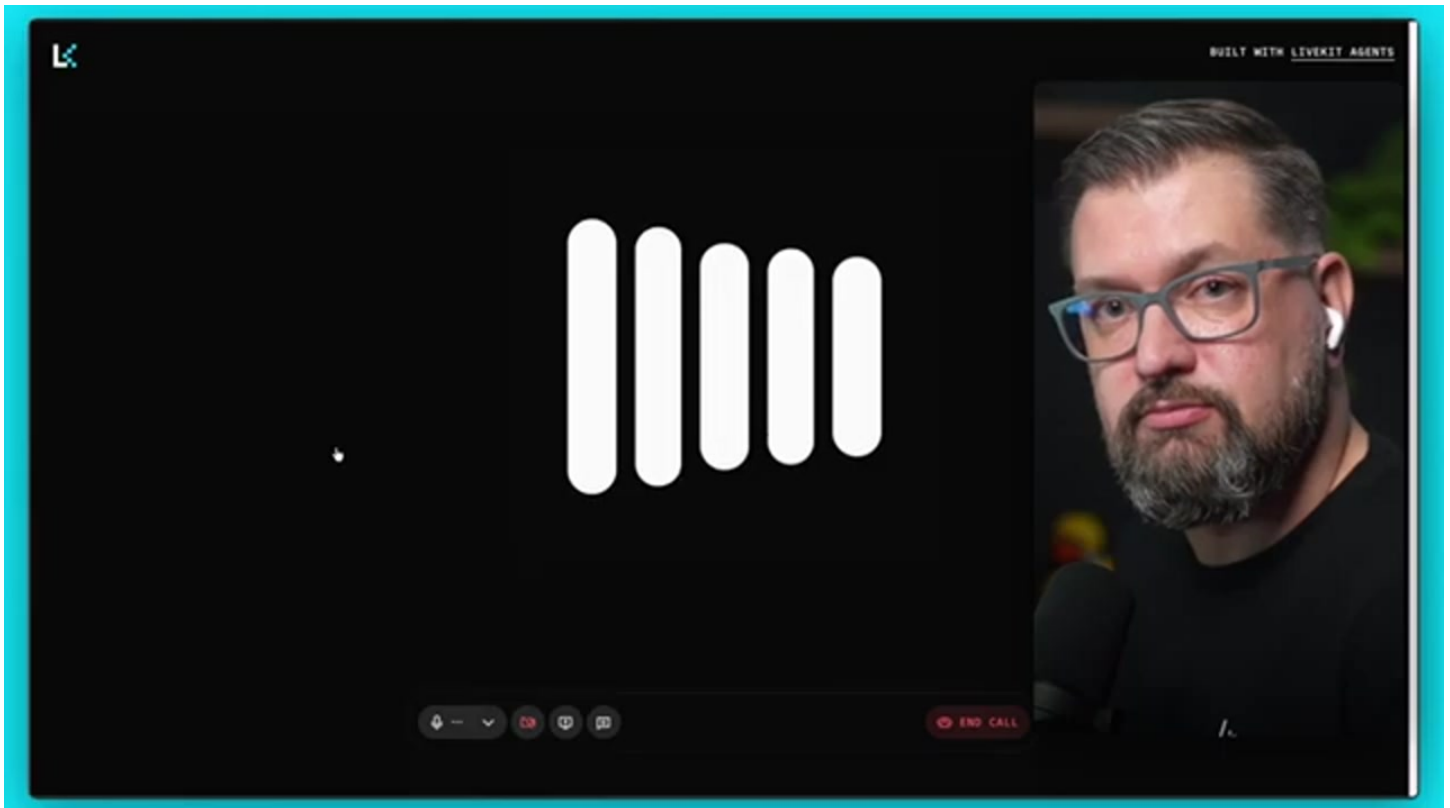
agents here. If we click on our agent, uh we can then see some more detailed information about our agent. But in order to test it, let's go over to sandbox and we're going to create a new sandbox. So we'll create a new web voice agent uh sandbox. And let's go ahead and create create this uh sandbox here. You can give it a name or just let it

■ [38:31 - 38:51]



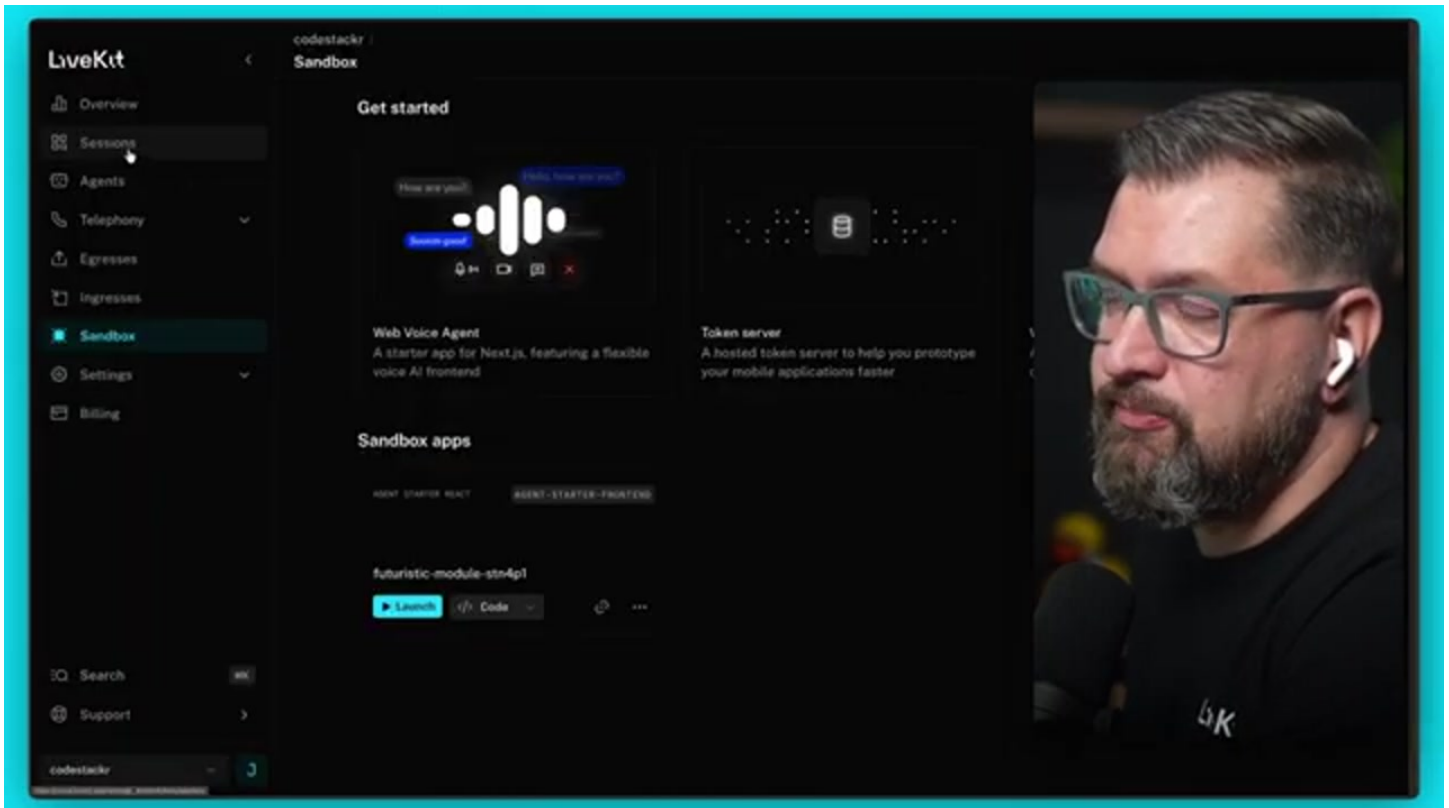
can give it a name or just let it randomly generate a name. Let's go ahead and create it. And then we don't need these instructions. Let's go ahead and launch this. It's going to use our agent. So, let's go ahead and click start call and we'll need to enable the microphone. Hello. >> Hey there. What tech trouble can I zap

■ [38:51 - 39:11]



>> Hey there. What tech trouble can I zap for you today? >> Uh, more tech trouble. Um, I'm actually not having much tech trouble. Uh, so thank you. We'll go ahead and end the call. And now we should have some observability data on that really brief call there. Uh, but you can see it was still the same agent that we created. uh the same voice asking about tech trouble. Let's go ahead and go back over to our dashboard and this time let's go

■ [39:11 - 39:31]



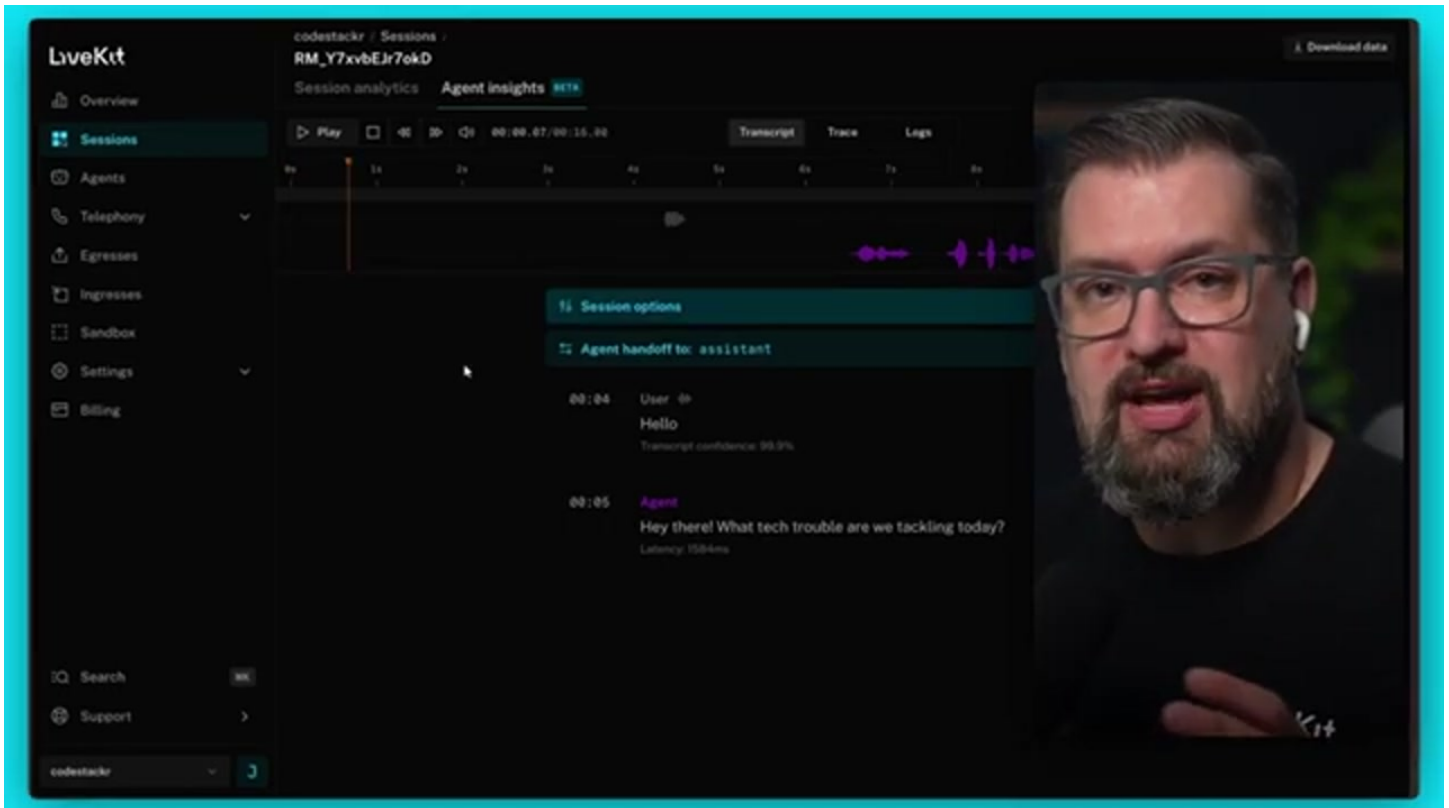
to our dashboard and this time let's go ahead and click on sessions and we can see our latest session here. We can click on it. We have this agent insights tab and this is where we can see our observability data and here is that timeline. So we can go ahead and click play here.

■ [39:31 - 39:51]

The screenshot displays the LiveKit web interface. On the left is a sidebar with navigation links: Overview, Sessions (active), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main area is titled 'codestackr / Sessions / RM_Y7xvbEjr7okD'. It features a 'Session analytics' tab and an 'Agent insights' tab (marked as BETA). Below these are controls for 'Pause', 'Play', 'Stop', and 'Reset', along with a timestamp '00:05:03/00:15:72'. A 'Transcript' tab is active, showing a conversation timeline. The transcript includes a 'Session options' section with 'Agent handoff to: assistant'. The conversation log shows a user saying 'Hello' at 00:03 with a transcript confidence of 99.9%, and an agent responding at 00:05 with 'Hey there! What tech trouble are we tackling today?' and a latency of 1564ms. An audio waveform is visible above the transcript. On the right, a video feed shows a man with glasses and a beard wearing a white earbud.

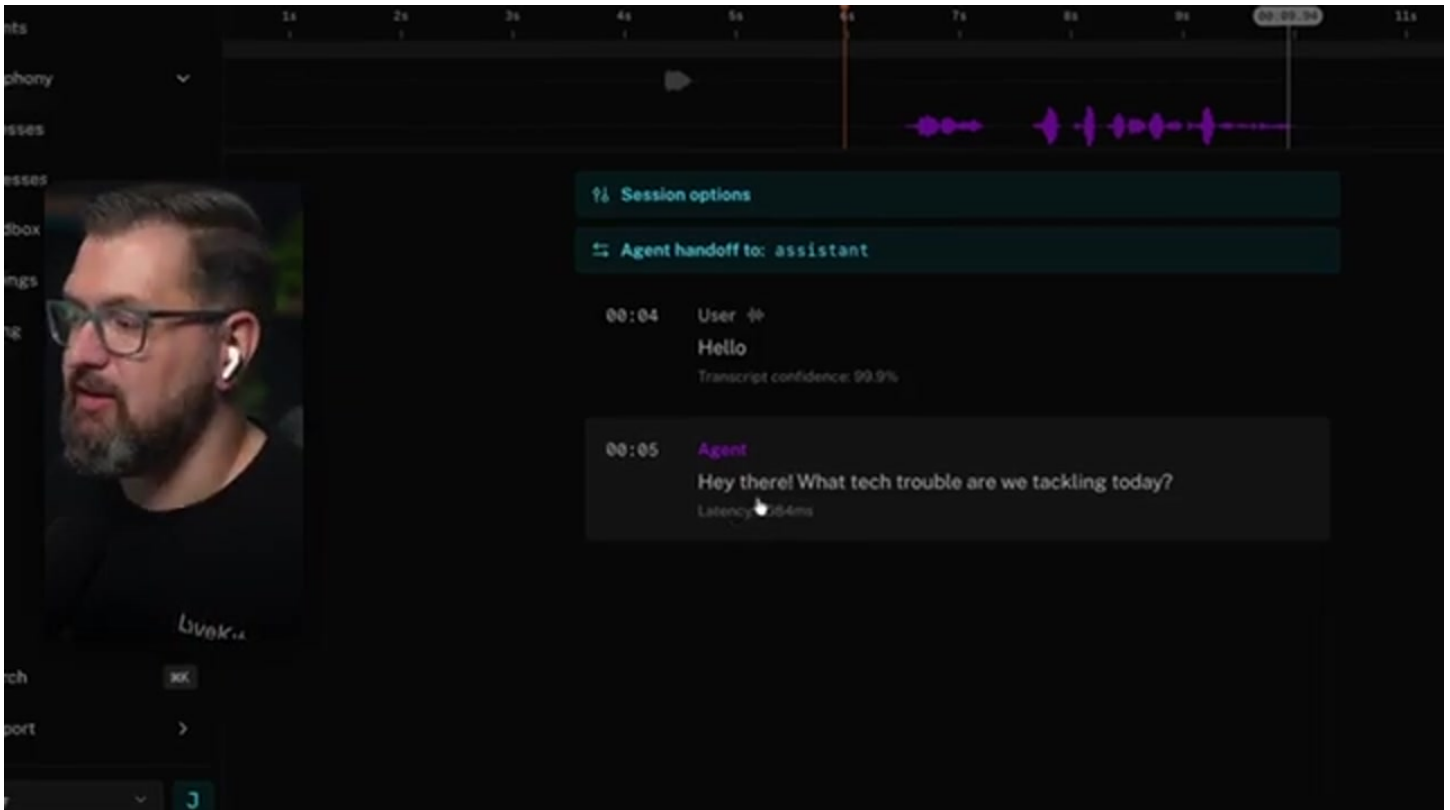
Hello. >> Hey there. What tech trouble are we tackling today? >> Looks like I ended it before my last response um actually made its way up. So, um I probably should have talked a little bit longer, but you can see it recorded my audio, the agent's audio. So, the transcript view shows the conversation timeline with audio

■ [39:51 - 40:11]



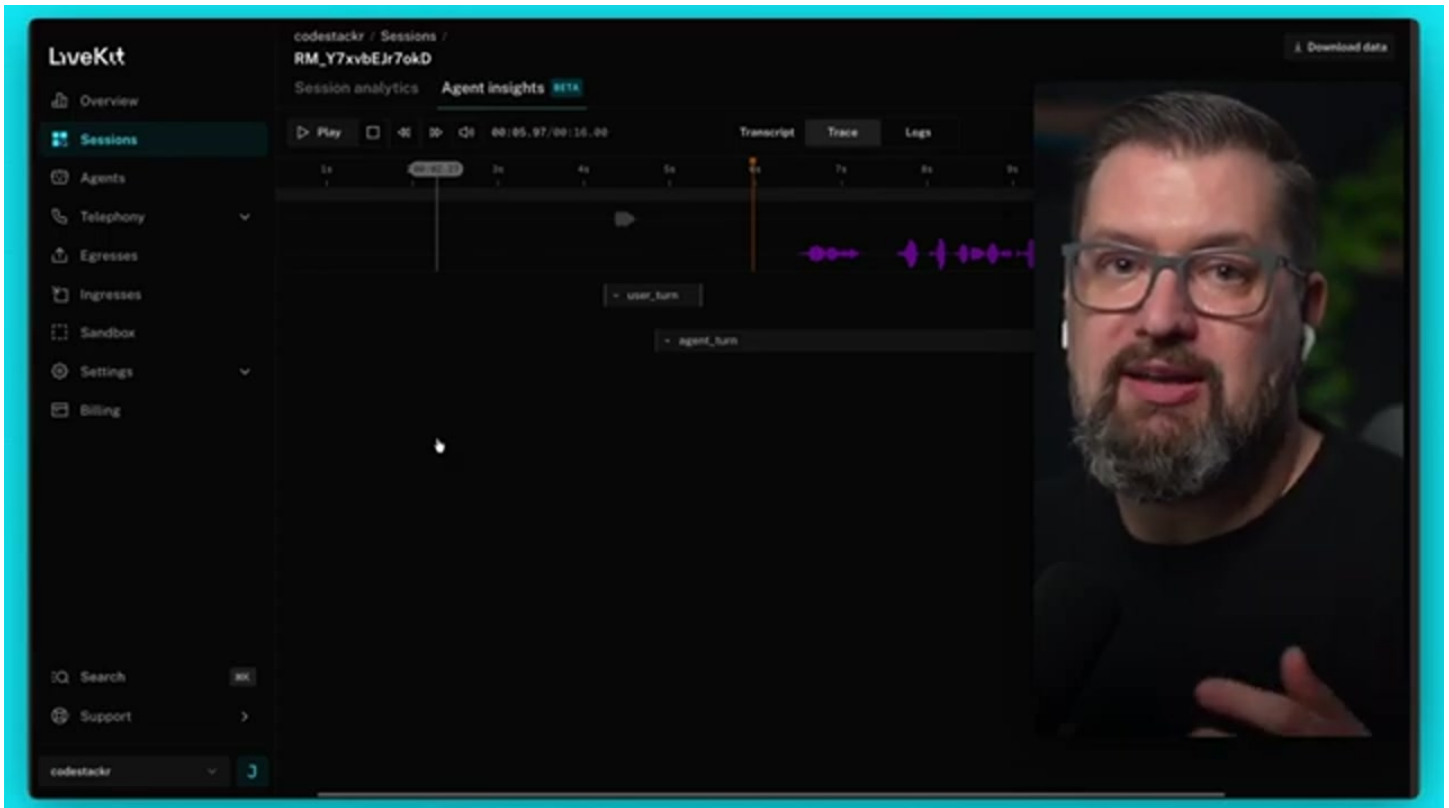
conversation timeline with audio playback. You can scrub to any point and hear exactly what the user uh and agent said. In inline alerts, highlight key events like tool calls. And this view is great for spotting interruptions and understanding the conversation flow. So we can see here that the agent latency was 1.5 seconds. If we click on it, we

■ [40:11 - 40:31]



was 1.5 seconds. If we click on it, we can see some more stats. Now the trace view is where you'll spend most of your optimization time. Each agent response breaks down into spans for ST, LLM, and TTS. You can click on any of these and see extra details, timestamps, and durations. And this is where you can find and identify bottlenecks. If your

■ [40:31 - 40:51]



find and identify bottlenecks. If your LLM span is consistently over 500 milliseconds, you might consider a faster model or enabling preemptive generation if you haven't already. Uh if TTS is the bottleneck, try a different voice provider. So here again, we can click on the agent turn, we can see the LLM node and we can get all this extra

■ [40:51 - 41:11]

The screenshot displays a voice AI interface. On the left, a video of a man with a beard and glasses is shown. The central timeline features a play button and a progress bar. On the right, a detailed log for the 'llm_node' is visible, including start and end times, duration, agent transcript, and a JSON object of attributes.

llm_node
Dec 26, 2025, 8:52:52.839 PM

Start time	End time
Dec 26, 2025, 8:52:52.839 PM	Dec 26, 2025, 8:52:53.632 PM

Duration: 793ms

Agent transcript
Hey there! What tech trouble are we tackling today?

Attributes

```
{
  "room_id": "RM_Y7xvbEJr7okD",
  "job_id": "AJ_Ggeh2XCeYbDR",
  "lk.chat_ctx": {
    "items": [
      {
        "id": "lk.agent_task.instructions",
        "type": "message",
        "role": "system",
        "content": [
          {
            "text": "You are an upbeat, slightly sarcastic voice AI for tech support. Help the user fix issues without..."
          }
        ]
      }
    ]
  },
  "interrupted": false,
  "created_at": 1766803968.6842406
}
```

1: {
 "id": "item_483a27b1219d"

LLM node and we can get all this extra information. Same with the TTS node and when the agent speaks, we get all of this uh extra details that can help us. So we can see exactly where time is spent in each turn. So in this example, we can see the LLM node is taking 793 milliseconds. The TTS node is taking almost 2 seconds. So that's right there.

■ [41:11 - 41:31]

The screenshot displays the LiveKit web interface. On the left is a sidebar with navigation links: Overview, Sessions (active), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main area is titled 'codestackr / Sessions / RM_Y7xvbEjr7okD'. Below this, there are tabs for 'Session analytics' and 'Agent insights' (which is selected). The 'Agent insights' tab shows a timeline with a play button, a stop button, and a search bar. Below the timeline, there are several data points: 'user_turn', 'agent_turn', 'lm_node', 'tts_node', and 'agent_speaking'. On the right side, there is a video feed of a man with glasses and a beard, wearing a black t-shirt with the 'LiveKit' logo. To the right of the video feed, there is a panel with session details: 'Start time: Dec 26, 2023', 'Duration: 1974ms', and 'Attributes' (a JSON object with 'room_id' and 'job_id').

almost 2 seconds. So that's right there. That's where we can start considering optimization. So going over to the logs view, uh this is where it will show information, warnings, errors, and debug messages from your agent code, the media server, and client connections. So if something fails or you see unexpected behavior in the transcript, click here

■ [41:31 - 41:51]

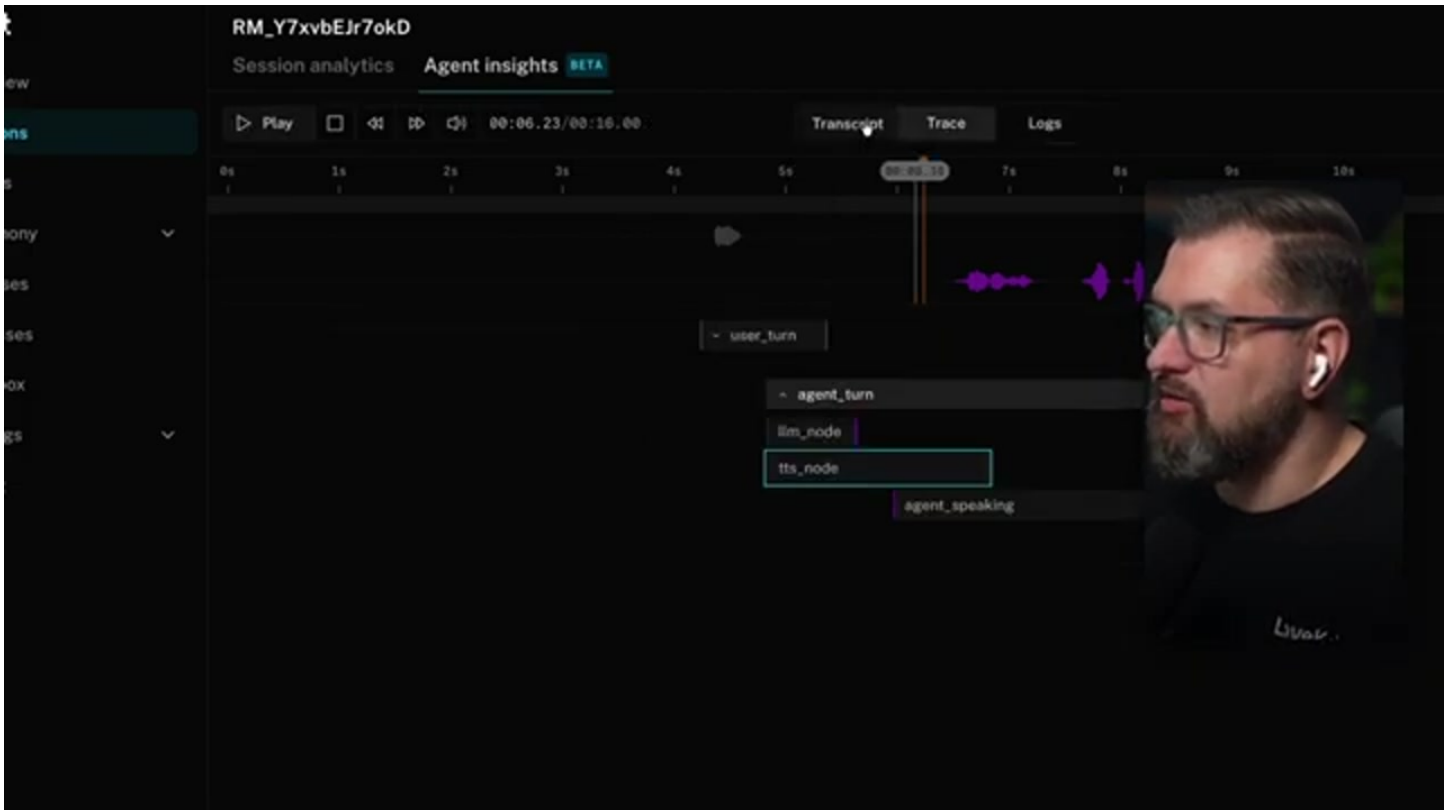
The screenshot displays the LiveKit dashboard for a session titled "RM_Y7xvbEjr7okD". The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for Overview, Sessions (active), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. At the bottom, there are links for Search, Support, and a user profile for "codestackr".
- Top Bar:** Shows the session ID "RM_Y7xvbEjr7okD" and a "Download data" button.
- Session Analytics:** Includes tabs for "Session analytics" and "Agent insights" (marked as BETA). Below these are controls for playing/pausing the audio and a timeline scrubber.
- Table:** A table with columns "Timestamp", "Level", "Logger", and "Message" showing various metrics and events.
- Video Feed:** A large video window on the right shows a man with glasses and a beard, wearing a black t-shirt with the "LiveKit" logo.

Timestamp	Level	Logger	Message
Dec 26, 00:52:52.837 PM	INFO	livekit.agents	STT metrics
Dec 26, 00:52:53.369 PM	INFO	livekit.agents	EOU metrics
Dec 26, 00:52:53.632 PM	INFO	livekit.agents	LLM metrics
Dec 26, 00:52:53.971 PM	INFO	...mp_main...	Time to first audio: 0.602s
Dec 26, 00:52:54.889 PM	INFO	livekit.agents	TTS metrics
Dec 26, 00:53:00.312 PM	INFO	livekit.agents	closing agent session due to participant disconnect id
Dec 26, 00:53:04.151 PM	INFO	...mp_main...	Usage summary: UsageSummary(tim_prompt_tokens=

behavior in the transcript, click here for the technical details. Again, we can click on each of these to show the extra information. Now, the really cool thing about observability here is that everything is all aligned to the same audio track. And so as we're scrubbing through the audio, we are in the same timeline looking at the same things in all of these uh all three of these

■ [41:51 - 42:11]



all of these uh all three of these different views. So they're all in sync. So if you have observability enabled at the project level, but you want to disable it for a specific session or for a specific agent, you can do that. So we'll just go down here to session start and then we can add an option here. We can add record equals false. Pretty

■ [42:11 - 42:31]

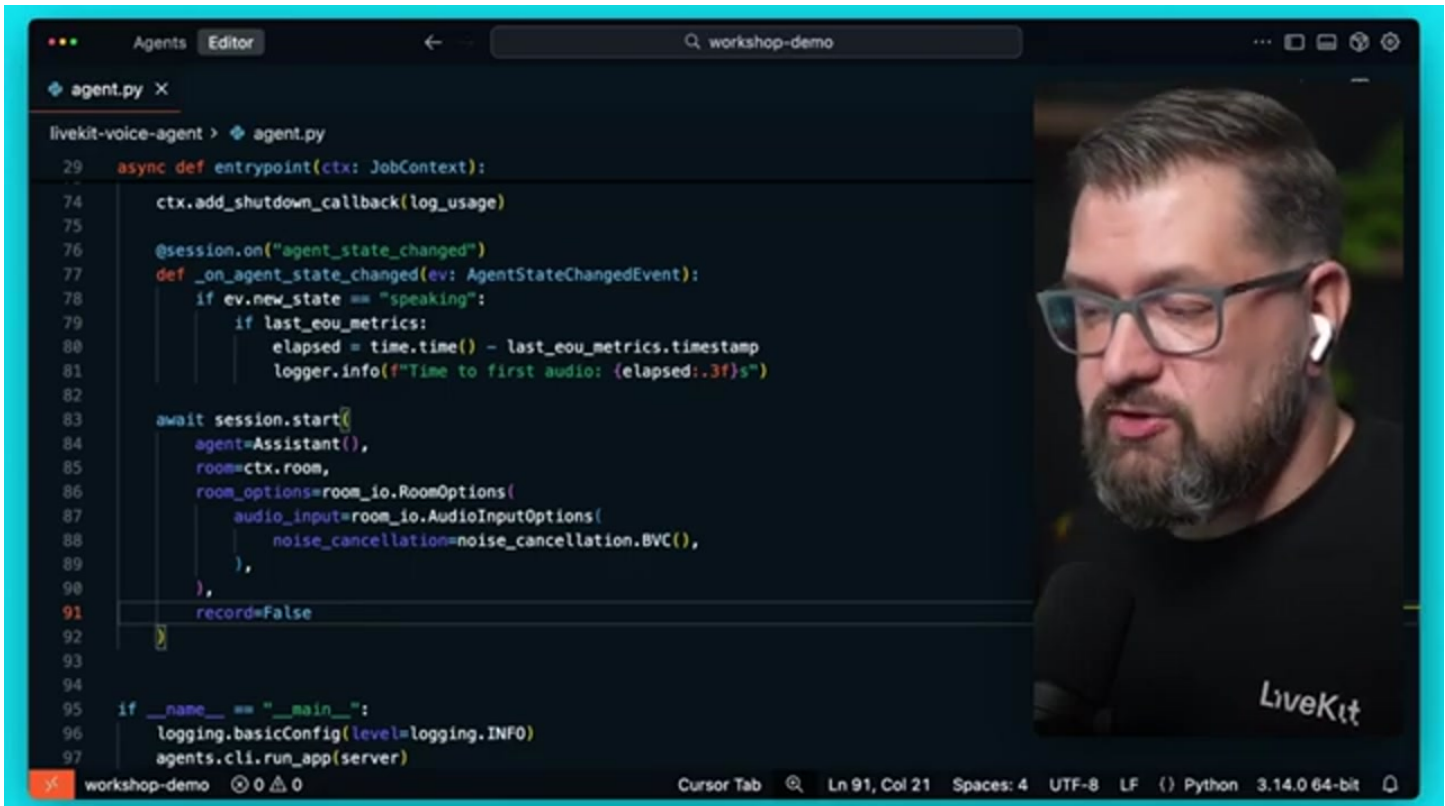
```
74     ctx.add_shutdown_callback(log_usage)
75
76     @session.on("agent_state_changed")
77     def _on_agent_state_changed(ev: AgentStateChangedEvent):
78         if ev.new_state == "speaking":
79             if last_eou_metrics:
80                 elapsed = time.time() - last_eou_metrics.timestamp
81                 logger.info(f"Time to first audio: {elapsed:.3f}s")
82
83         await session.start(
84             agent=Assistant(),
85             room=ctx.room,
86             room_options=room_io.RoomOptions(
87                 audio_input=room_io.AudioInputOptions(
88                     noise_cancellation=noise_cancellation.BVC(),
89                 ),
90             ),
91             record=False
92         )
93
94
95 if __name__ == "__main__":
96     logging.basicConfig(level=logging.INFO)
97     agents.cli.run_app(server)
```



workshop-demo 0 0 Cursor Tab Ln 91, Col 21 Spaces: 4 UTF-8

can add `record equals false`. Pretty simple. And now this agent will no longer collect observability data. So this disables the uh upload of audio transcripts, traces and logs for the entire session even if the project level setting is enabled. So use this when you need to handle sensitive data or you want to reduce storage costs for specific sessions. Now all observability

■ [42:31 - 42:51]



The screenshot shows a video call window. On the left, a code editor displays the `agent.py` file. The code is written in Python and includes an `async def entrypoint` function. It sets up a session with an `Assistant` agent, configures room options for audio input and noise cancellation, and starts the session. A `_on_agent_state_changed` event listener is also defined to log the time to first audio when the agent starts speaking. The right side of the window shows a video feed of a man with a beard and glasses, wearing a black t-shirt with the `LiveKit` logo. The status bar at the bottom indicates the current position in the code is at line 91, column 21, and the file is encoded in UTF-8.

```
29 async def entrypoint(ctx: JobContext):
74     ctx.add_shutdown_callback(log_usage)
75
76     @session.on("agent_state_changed")
77     def _on_agent_state_changed(ev: AgentStateChangedEvent):
78         if ev.new_state == "speaking":
79             if last_eou_metrics:
80                 elapsed = time.time() - last_eou_metrics.timestamp
81                 logger.info(f"Time to first audio: {elapsed:.3f}s")
82
83     await session.start(
84         agent=Assistant(),
85         room=ctx.room,
86         room_options=room_io.RoomOptions(
87             audio_input=room_io.AudioInputOptions(
88                 noise_cancellation=noise_cancellation.BVC(),
89             ),
90         ),
91         record=False
92     )
93
94
95 if __name__ == "__main__":
96     logging.basicConfig(level=logging.INFO)
97     agents.cli.run_app(server)
```

specific sessions. Now all observability data is stored in the US and is retained for 30 days and data older than 30 days is automatically deleted. Now you can download the audio transcripts and logs directly from the session if you need a copy. Now for projects on the free build plan, some uh anonymized session data may be retained longer for model

Data retention

- 30-day retention window
- Automatic deletion after 30 days
- Data stored in the US



LIVEKIT.IO

may be retained longer for model improvement purposes. Uh paid plans including ship scale and enterprise have full data deletion after that 30-day window. A low latency voice requires parallel work across the stack. So here's some best practices. First, stream everything. ST LLM TTS should operate incrementally, not in big

Latency optimization

— Stream everything



LIVEKIT.IO

operate incrementally, not in big batches. Second, parallelize. Start uh TTS as soon as the LLM emits the first words. Don't wait for the full sentence. Third, avoid blocking IO. Keep tool calls and storage async. Bound your timeouts and retries. And fourth, keep prompts tight. Fewer tokens means faster

Latency optimization

- Stream everything
- Parallelize STT → LLM → TTS
- Avoid blocking I/O
- Keep prompts tight



prompts tight. Fewer tokens means faster first audio and lower cost. Now, the metrics you collect at the application level only tell you part of the story. Network conditions also affect perceived latency. WebRTC handles this with Opus audio compression, jitter buffering, and adaptive bit rate. If you see unexpectedly high time to first audio

Latency optimization

- Stream everything
- Parallelize STT → LLM → TTS
- Avoid blocking I/O
- Keep prompts tight



LIVEKIT.IO

unexpectedly high time to first audio that doesn't correlate with your pipeline metrics, network conditions may be the culprit. Use agent observability to validate and and fine-tune agent performance. The trace view shows exactly where time is spent in each turn. So, this makes it easier to identify bottlenecks. So to recap, these

Latency optimization

- Stream everything
- Parallelize STT → LLM → TTS
- Avoid blocking I/O
- Keep prompts tight



LIVEKIT.IO

identify bottlenecks. So to recap, these are the metrics worth tracking as you continue building. You've seen them in the console output during development and in the trace view in production. And so aim for a time to first audio under 1 second in most cases. And use the trace breakdown to identify which component to optimize when you're over budget. So

Key metrics

- TTFT (Time to First Token)
- TTFA (Time to First Audio)
- Interruption rate
- Tool latency
- Fallback activation
- Target: <1000ms



LIVEKIT.IO

optimize when you're over budget. So you've learned which metrics matter for voice agents and how to collect them locally. You've improved latency with preemptive generation. You've deployed your agent to LiveKit Cloud, and you've seen how agent observability uh gives you transcripts, traces, and log views to debug and optimize each session. With

- ✓ Key metrics
- ✓ Usage collection
- ✓ Preemptive generation
- ✓ LiveKit Cloud deployment
- ✓ Agent observability



LIVEKIT.IO

to debug and optimize each session. With metrics and observability in place, you can make datadriven decisions as you add tools and workflows in the next lesson.